

Air Lines Workforce, Operations, and Financial Performance Analysis

Sareena Gurung

TABLE OF CONTENTS

Goal	2
Research/Business Goal	2
Queries	2
Sources	2
Collect Data.....	3
Flight Statistics.....	3
Employee Data.....	9
Financial Data	15
Investigating for Issues	22
Flight Statistics.....	22
Employee Data.....	22
Financial Data	23
Mismatched Reporting Cadence.....	24
Outliers.....	24
Cleaning Data	24
Flight Statistics.....	24
Employee Data.....	26
Financial Data	27
Mismatched Reporting Cadence.....	28
Outliers.....	28
Analysis	30
Data Transformation and Integration	30
Why the Research Queries Were Not Fully Addressed	32
Conclusion.....	33
References.....	34
Appendix A.....	35
Code	35
Appendix B	51
Delta-Filtered Outlier Summary (IQR Method).....	51

GOAL

Research/Business Goal

- Measure the impact of Delta's employment levels on peak-season flight cancellations and financial performance.

Queries

- What is Delta Air Lines' flight cancellation rate during peak travel seasons?
- How does employment headcount and employment level (FT vs. PT) change over time?
Are there seasonal changes?
- Can we utilize employee headcount to predict flight cancellations or vice versa?
- Can we utilize employee headcount to predict financial performance or vice versa?

Sources

- Delta Airline Flight Performance Statistics
 - Monthly flight performance statistics such as flight count and cancellation rates.
 - Sourced from the U.S. Department of Transportation's BTS.
- Delta Employment Data:
 - Yearly employment data such as staffing count and employment level.
 - Sourced from the U.S. Department of Transportation's BTS.
- Delta Financial Data:
 - Quarterly financial data.
 - Sourced from the U.S. Department of Transportation's BTS and from Delta Air Lines' investor relations website.

COLLECT DATA

Three data sources – Flight Statistics, Employee Data, and Financial Data – are to be established as the primary data marts to answer the business goal.

Flight Statistics

- **Title:** USA Airlines Flight Performance Statistics (2015-2025)
- **Sources:**
 - Creator: Bureau of Transportation Statistics
 - Link: <https://www.transtats.bts.gov/ontime/Airline.aspx>
- **Past Usage:**
 - The Bureau of Transportation Statistics publishes flight data monthly to inform the public about airline service performance — including delays, cancellations, and diversions. It's part of official quality reporting under federal government regulations.
- **Relevant Information Paragraph:**
 - The dataset was compiled by the Bureau of Transportation Statistics (BTS) and contains monthly aggregated airline performance data for all United States airlines. The data spans from January 2015 through November 2025, with observations organized by year and month.
 - The dataset includes quantitative performance measures (recorded in minutes) related to both departures and arrivals.
 - Departure time metrics include departure delay, taxi-out time, and take-off time
 - Arrival time metrics include arrival delay, airborne time, and taxi-in time
 - In addition, the dataset includes summative operational measures such as:
 - Cancelled flight count
 - Diverted flight count
 - Late flight count
 - These attributes allow for performance analysis and trend evaluation to support the proposed business objective.
- **Types of Data and Measurement Scales:** Refer to #8
- **Number of Instances:** 2135
- **Number of Attributes:** 31
- **Attribute Values:**

Attribute	Scale	Values	Missing	Outliers *
Airline_Nameairline	Nominal	20 unique values (Alaska Airlines Inc., Allegiant Air, etc.)	None	N/A
Airline_Code	Nominal	20 unique values (AS, G4, etc.)	None	N/A
Month	Ordinal	12 unique values (1-12)	None	N/A
Year	Interval	10 unique values (2015-2025)	None	N/A
All_Total Number	Ratio	Min: 1283.00 Max: 128588.00	None	78

All_Average Departure Delay (minutes)	Ratio	Min: 9.07 Max: 49.18	None	40
All_Average Taxi-Out Time (minutes)	Ratio	Min: 9.13 Max: 24.39	None	None
All_Average Scheduled Departure to Takeoff (minutes)	Ratio	Min: 32.23 Max: 41.13	None	52
All_Average Arrival Delay (minutes)	Ratio	Min: 24.52 Max: 45.56	None	61
All_Average Airborne Time (minutes)	Ratio	Min: 56.05 Max: 195.59	None	None
All_Average Taxi-In Time (minutes)	Ratio	Min: 4.03 Max: 13.94	None	43
All_Total Number Cancelled	Ratio	Min: 0.00 Max: 48707.00	None	168
All_Percent Flights Cancelled	Ratio	Min: 0.00 Max: 84.68	None	166
All_Total Number Diverted	Ratio	Min: 0.00 Max: 545.00	None	150
All_Percent Flights Diverted	Ratio	Min: 0.00 Max: 2.06	None	53
Late_Total Number	Ratio	Min: 20.00 Max: 39050.00	None	135
Late_Average Departure Delay (minutes)	Ratio	Min: 20.78 Max: 113.63	None	9
Late_Average Taxi-Out Time (minutes)	Ratio	Min: 10.36 Max: 37.79	None	5
Late_Average Scheduled Departure to Take-off (minutes)	Ratio	Min: 63.77 Max: 99.93	None	62
Late_Average Arrival Delay (minutes)	Ratio	Min: 27.87 Max: 115.80	None	8
Late_Average Airborne Time (minutes)	Ratio	Min: 60.53 Max: 267.50	None	4
Late_Average Taxi-In Time (minutes)	Ratio	Min: 4.63 Max: 17.30	None	44
Late_Percent Flights Late	Ratio	Min: 4.63 Max: 17.30	None	44
Cause_Total Flights	Ratio	Min: 1283.00 Max: 128588.00	None	78
Cause_Diverted Flights Reaching Scheduled Destination	Ratio	Min: 0.00 Max: 545.00	None	150

Cause_Diverted Flights	Ratio	Min: 0.00 Max: 175.00	None	44
Cause_Cancelled Flights	Ratio	Min: 0.00 Max: 200.00	None	52
Cause_Weather	Ratio	Min: 0.00 Max: 24.14	None	93
Cause_National Aviation System	Ratio	Min: 0.00 Max: 71.65	None	70
Cause_Security	Ratio	Min: 0.00 Max: 3.89	None	134
Cause_Late Aircraft Arrival	Ratio	Min: 1.03 Max: 666.58	None	25

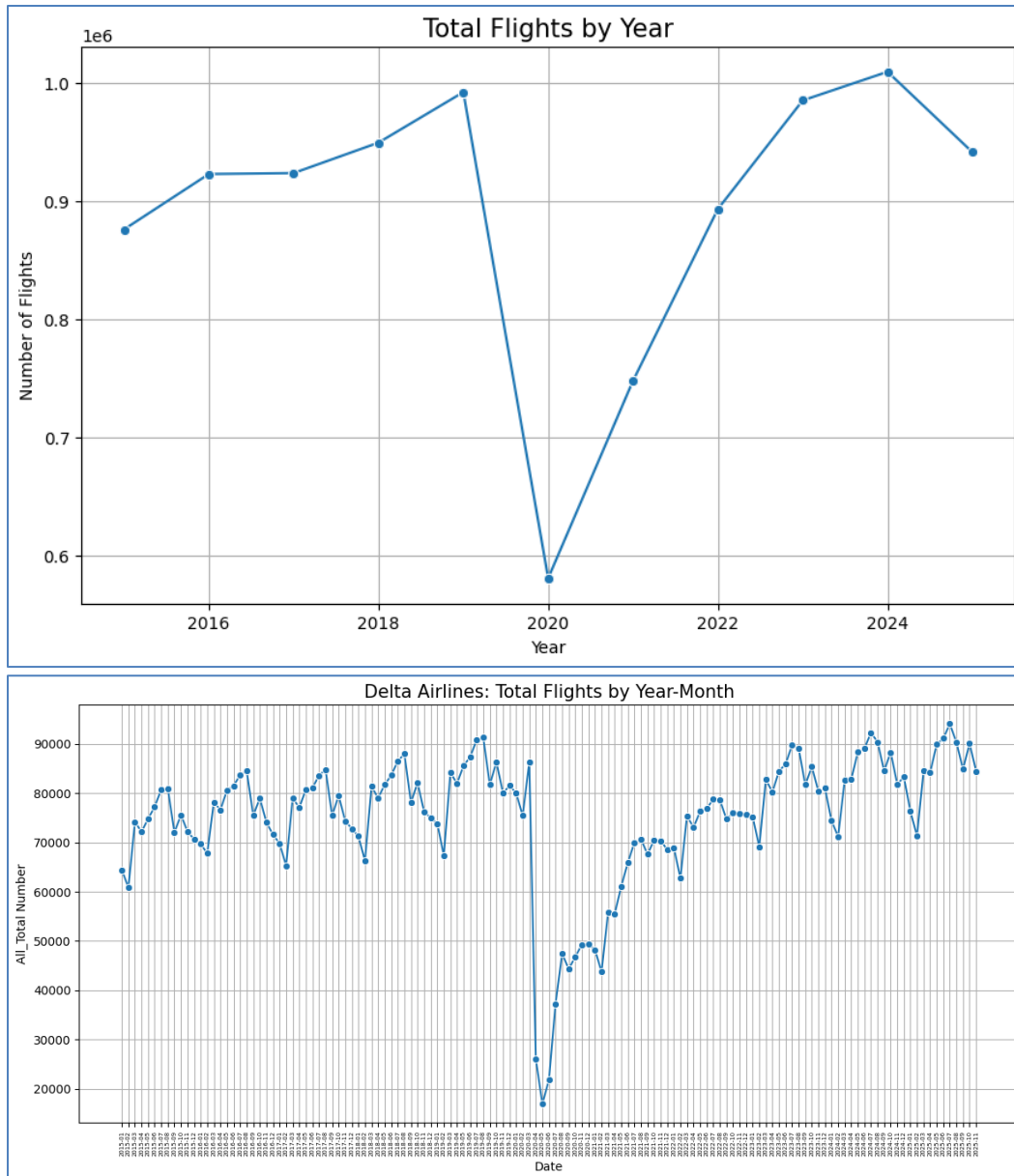
* Outliers are defined as values beyond 1.5 times the Interquartile range (IQR) (Tukey, 1977).

- **Missing Attribute Values:** Refer to #8
- **Outliers using IQR method:** Refer to #8
- **Class Distribution:**

Class (Year)	N (Percent) *
2015	163 (7.63%)
2016	144 (6.74%)
2017	144 (6.74%)
2018	224 (10.49%)
2019	222 (10.40%)
2020	219 (10.26%)
2021	210 (9.84%)
2022	210 (9.84%)
2023	209 (9.79%)
2024	204 (9.56%)
2025	186 (8.71%)

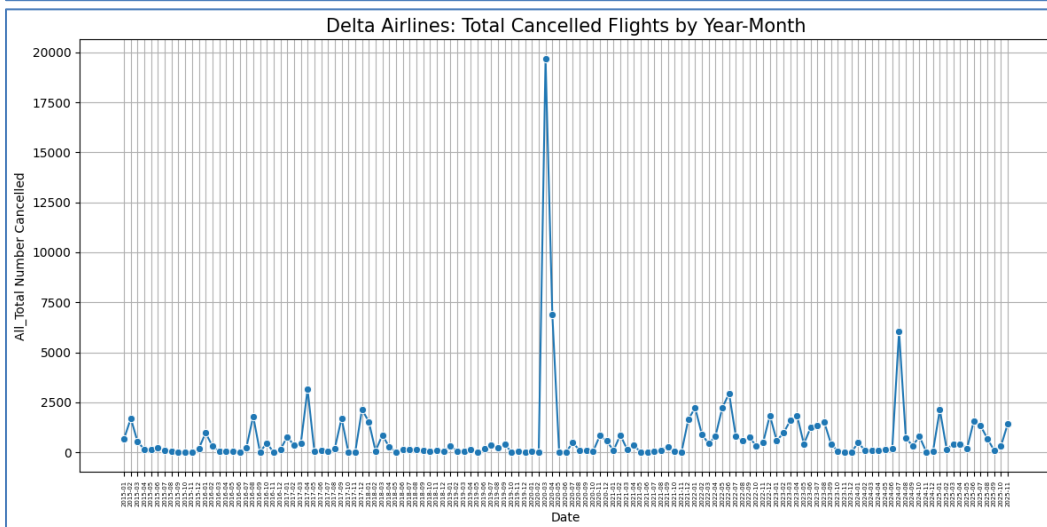
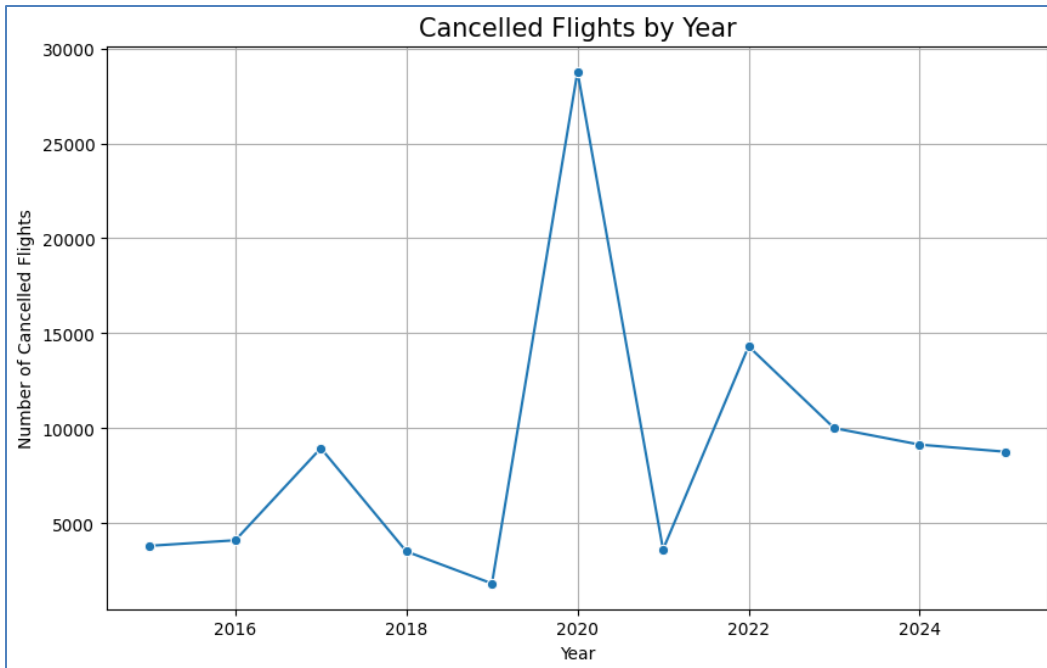
*131 (12 per year except for 2025 with only 11) instances are for Delta Air Lines Inc.

- **Analysis:**
The data primarily consists of quantitative data. A descriptive statistical analysis approach for Delta Airlines is provided below.
 - Total Flights: All_Total Number



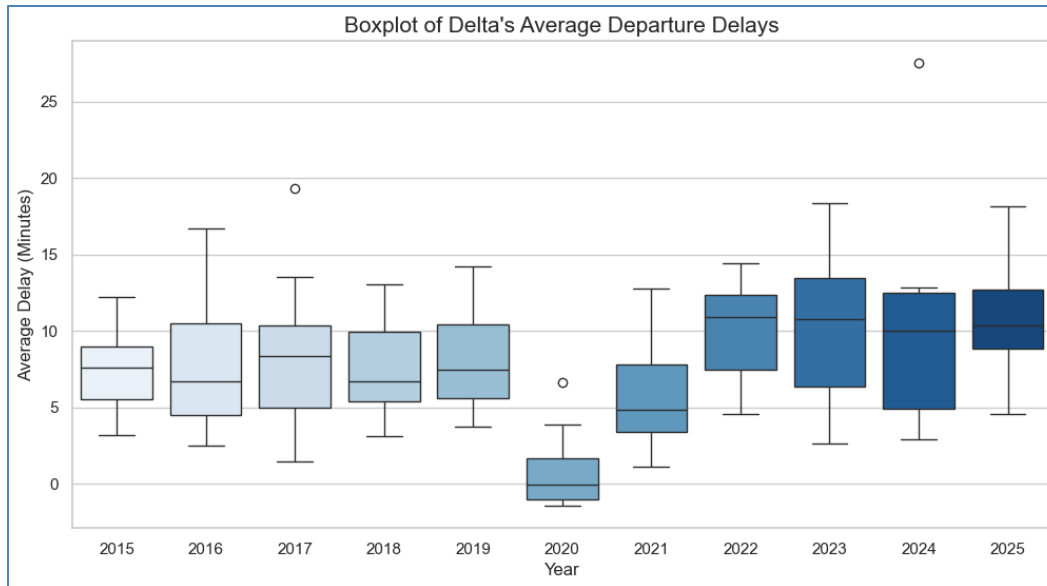
Delta's flight count shows a significant disruption in 2020, where annual flight volume plummeted to below 600K flights, a 41% decrease compared to the 2019 peak of 992K flights. While a recovery began in August 2020, flights did not return to pre-COVID levels until 2023 (995K flights). Notably, 2024 shows the highest volume in the dataset at 1 million flights, suggesting a full market return before a slight drop in late 2025.

- Cancelled Flights: All Total Number Cancelled



Delta's yearly cancellations surged to 29K, a 1,461% increase compared to the 2019 low of just 1.8K. While 2021 saw a brief return to stability, another drop came in 2022 as cancellations spiked back up to 14K flights. Notably, while the trend has cooled since 2023, roughly 9K annual cancellations remain significantly higher than pre-COVID levels.

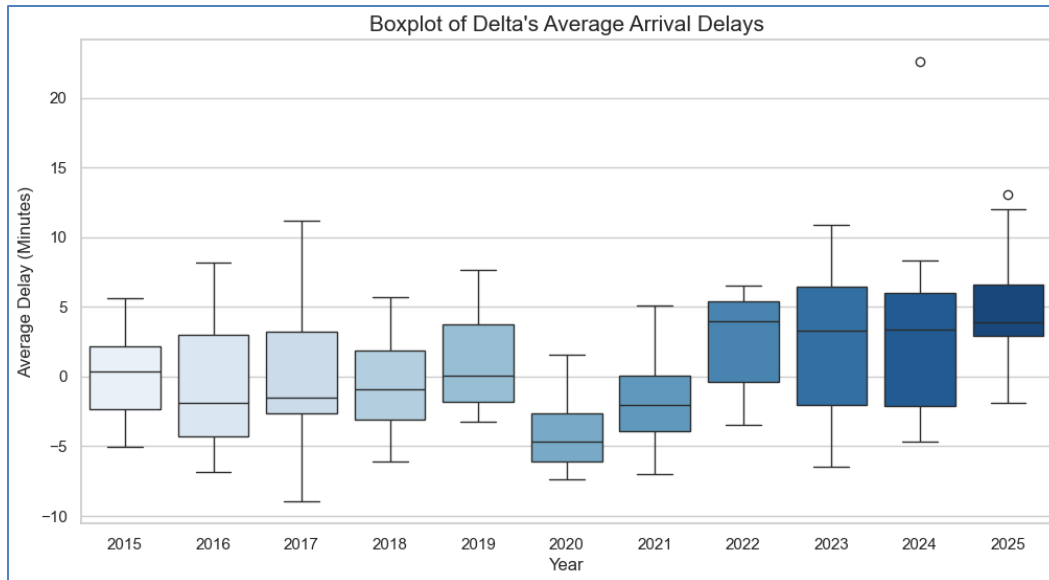
- Departure Delay: All_Average Departure Delay (minutes)



Year	Mean	Median	Min	Max	Std Dev	Count	IQR
2015	7.41	7.61	3.21	12.22	2.88	12	3.40
2016	7.84	6.74	2.54	16.69	4.41	12	6.02
2017	8.50	8.38	1.47	19.35	4.83	12	5.41
2018	7.38	6.70	3.16	13.04	3.09	12	4.52
2019	8.11	7.50	3.75	14.20	3.34	12	4.83
2020	0.77	-0.05	-1.43	6.65	2.46	12	2.64
2021	5.62	4.87	1.12	12.78	3.46	12	4.44
2022	9.93	10.88	4.57	14.42	3.29	12	4.95
2023	10.30	10.78	2.66	18.35	5.15	12	7.06
2024	10.09	10.00	2.92	27.50	6.66	12	7.58
2025	11.06	10.36	4.59	18.13	4.06	11	3.88

Delta's departure delay data reveals volatility, most notably in 2020 where medians plummeted to an all-time low of -0.05 minutes due to COVID stopping flights. However, the post-pandemic recovery introduced a new era of inefficiency, with annual mean delays climbing from 5.62 minutes in 2021 to a peak of 11.06 minutes in 2025. Notably, 2024 stands out at a record-high spike of 27.50 minutes and the highest IQR (7.58).

- Arrival Delay: All_Average Arrival Delay (minutes)



Year	Mean	Median	Min	Max	Std Dev	Count	IQR
2015	0.23	0.40	-5.01	5.61	3.24	12	4.50
2016	-0.66	-1.86	-6.85	8.17	4.82	12	7.34
2017	-0.15	-1.52	-8.97	11.23	5.43	12	5.88
2018	-0.44	-0.90	-6.12	5.70	3.44	12	4.98
2019	1.11	0.04	-3.26	7.63	3.57	12	5.53
2020	-4.05	-4.69	-7.38	1.57	2.73	12	3.48
2021	-1.83	-2.02	-6.96	5.10	3.39	12	3.97
2022	2.55	3.99	-3.48	6.55	3.54	12	5.85
2023	2.36	3.28	-6.49	10.93	6.03	12	8.48
2024	3.49	3.38	-4.68	22.60	7.48	12	8.17
2025	5.09	3.89	-1.86	13.05	4.37	11	3.68

Delta's arrival delay data also shows volatility, with 2020 recording an all-time low median of -4.69 minutes as lower flights due to COVID allowed for earlier arrivals. However, the post-pandemic recovery shifted toward increased arrival delays, with annual mean delays climbing from -1.83 minutes in 2021 to a record peak of 5.09 minutes in 2025. Notably, 2024 shows instability with a massive record-high spike of 22.60 minutes, further confirming that while flight volumes have returned, the ability to maintain a punctual schedule has significantly declined compared to the pre-2020 era.

Employee Data

1. **Title:** USA Airlines Employee Database (2014-2024)

2. **Source:** U.S. Department of Transportation's BTS

- Link: <https://www.transtats.bts.gov/employment/>

3. **Past Usage:**

- Any airline that has the operational capacity to carry a combined 18,000 lbs between passengers, cargo, and fuel must report their monthly statistics on full- and part-time employees to the Bureau of Transportation Statistics. This data is often utilized for a wide variety of purposes, but the primary usage is an economic indicator and can assist in infrastructure decisions. Notably, the Bureau of Transportation Statistics utilized this data in their *Journal of Transportation and Statistics* which ran from 1998 to 2014.

4. **Relevant Information Paragraph:**

- Employee Database was extracted from the U.S. Department of Transportation's BTS's website for the years 2014 to 2024. The original data contained yearly employment data on staffing and employment numbers.

5. **Types of Data and Measurement Scales:** Refer to #8

6. **Number of Instances:** 1047

7. **Number of Attributes:** 19

8. **Attribute Values:**

Attribute	Scale	Values	Missing	Outliers *
Carrier Name	Nominal	79 unique values (21 Air LLC, ABX Air Inc, etc.)	None	N/A
Carrier's Operation Region	Nominal	Values: A, D, I, L, P, S	None	N/A
Year	Interval	Min: 2014 Max: 2024	None	N/A
General Managers	Ratio	Min: 0 Max: 3015	None	160
Pilots and Copilots	Ratio	Min: 0 Max: 10960	None	88
Other Flight Personnel	Ratio	Min: 0 Max: 6338	None	170
Passenger/General Services & Administration	Ratio	Min: 0 Max: 19373	None	138
Maintenance Employees	Ratio	Min: 0 Max: 10540	None	104
Aircraft Traffic Handling Group 1 Employees	Ratio	Min: 0 Max: 624	None	182
General Aircraft Traffic Handling Employees	Ratio	Min: 0 Max: 10679	None	199
Aircraft Control Employees	Ratio	Min: 0 Max: 1622	None	122

Passenger Handling Employees	Ratio	Min: 0 Max: 18369	None	216
Cargo Handling Employees	Ratio	Min: 0 Max: 14688	None	194
Trainees and Instructor	Ratio	Min: 0 Max: 1915	None	120
Statistical Employees	Ratio	Min: 0 Max: 10447	None	158
Traffic Soliciters	Ratio	Min: 0 Max: 1430	None	172
Other Employees	Ratio	Min: 0 Max: 15777	None	158
Transport Related Employees	Ratio	Min: 0 Max: 192348	None	127
Total Employees	Ratio	Min: 0 Max: 217469	None	160

**** Outliers are defined as values beyond 1.5 times the Interquartile range (IQR) (Tukey, 1977).**

9.Missing Attribute Values: Refer to #8

10.Outliers using IQR method: Refer to #8

11.Class Distribution: (number of instances by year for all airlines meeting the criteria specified by Bureau of Transportation Statistics)

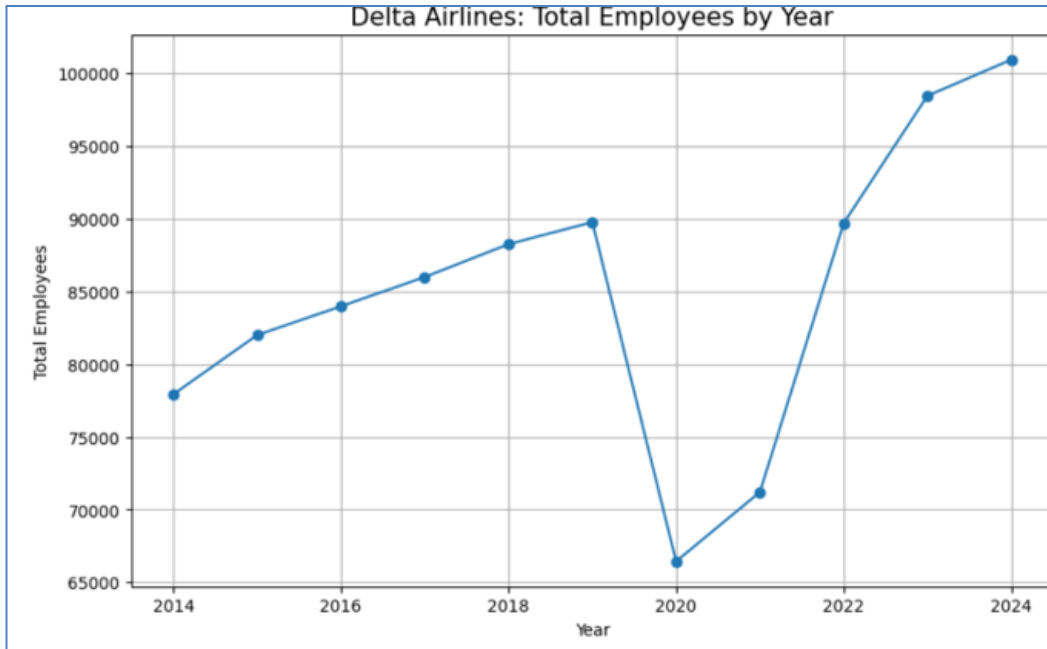
Class (Year)	N (Percent) *
2014	92 (8.79%)
2015	96 (9.17%)
2016	104 (9.93%)
2017	94 (8.98%)
2018	93 (8.88%)
2019	94 (8.98%)
2020	93 (8.88%)
2021	91 (8.69%)
2022	96 (9.17%)
2023	97 (9.26%)
2024	97 (9.26%)

*** From this data, 44 (4 per year) instances are for Delta Air Lines Inc.**

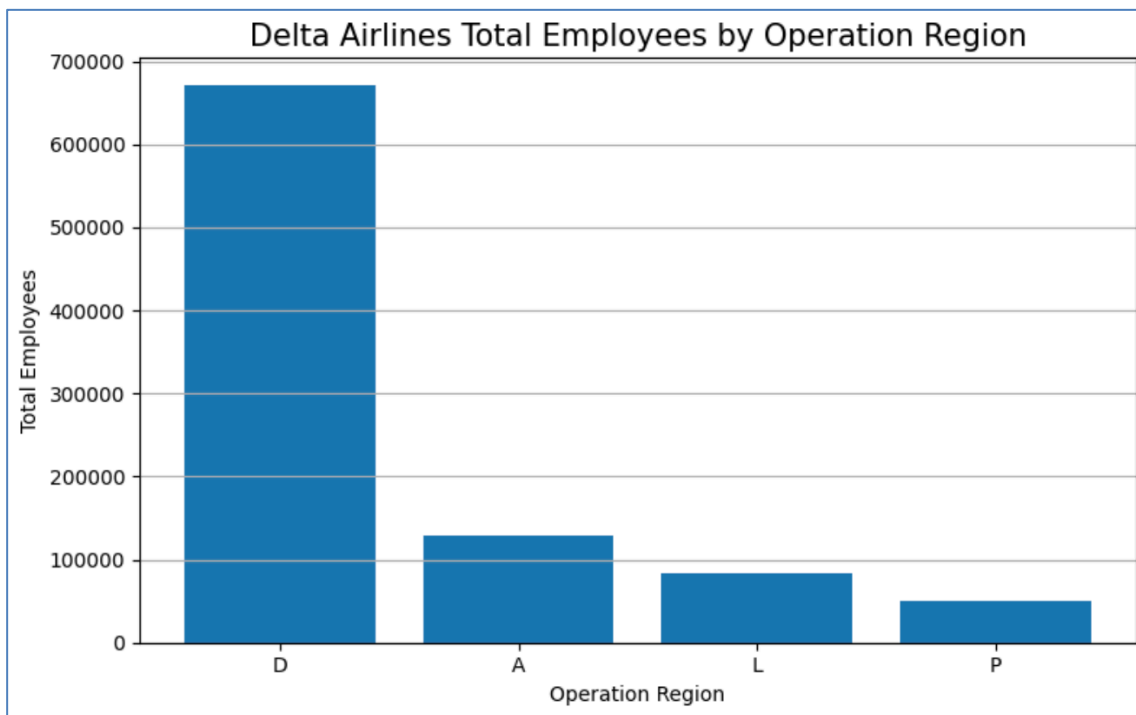
12.Analysis:

A descriptive statistical analysis approach for Delta Airlines is provided below.

- Total Employees by Year

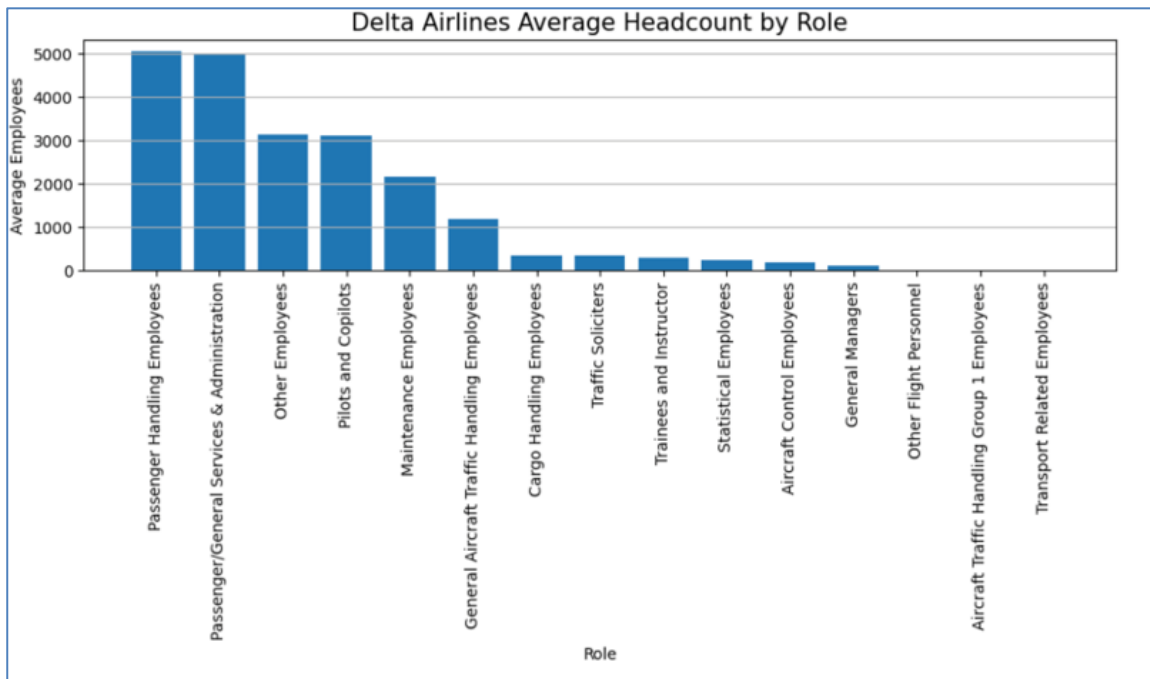


- Total Employees by Operating Region



D: Domestic (N=670573), A: Atlantic (N=129874), L: Latin America (N=84161), P: Pacific (N=49989)

- Average Headcount by Role



Role	Average Employees
Passenger Handling Employees	5064.20
Passenger/General Services & Administration	5017.00
Other Employees	3143.64
Pilots and Copilots	3108.70
Maintenance Employees	2170.30
General Aircraft Traffic Handling Employees	1180.86
Cargo Handling Employees	362.64
Traffic Solicitors	361.73
Trainees and Instructor	285.68
Statistical Employees	239.82
Aircraft Control Employees	193.43
General Managers	112.84
Other Flight Personnel	0.00
Aircraft Traffic Handling Group 1 Employees	0.00
Transport Related Employees	0.00

Financial Data

1. **Title:** Financial Data Mart

2. **Sources:**

- Publisher: U.S. Department of Transportation (DOT), Bureau of Transportation Statistics (BTS) - TranStats (Form 41 Financial Reports) Schedule P6
Link: https://www.transtats.bts.gov/Fields.asp?gnoyr_VQ=FME
- Supporting context source: Delta Air Lines Investor Relations - Financials (for performance context and cross-checking, as needed)
Link: <https://ir.delta.com/financials/>

3. **Past Usage:**

The raw TranStats table contains many fields beyond the needs of this project. For this project, only relevant fields are extracted from the raw source:

- Carrier and time identifiers needed to join across data marts (Carrier, Year, Quarter)
- Operating expense line-items (labor-related, fuel, materials, services, depreciation/amortization, other)
- Total operating expenses and major roll-ups (e.g., Transport Related Expense, Total Services, Total Materials).

4. **Relevant Information Paragraph:**

This Financials data mart provides quarterly operating expense components that can be aligned to staffing and flight performance outcomes. It supports relevant project goal analyses such as:

- Can we utilize employee headcount to predict flight cancellations or vice versa?
- Can we utilize employee headcount to predict financial performance or vice versa?

5. **Types of Data and Measurement Scales:** Refer to #8

6. **Number of instances:** 4676 (Quarterly records by carrier: (Carrier) x (Year) x (Quarter))

7. **Number of attributes:** 38

8. **Attribute Values:**

Attribute	Scale	Values *	Missing	Outliers **
General Management Personnel	Ratio	Min: -2380.46 Max: 50440.00	4 (0.09%)	581
Flight Personnel	Ratio	Min: -133.11 Max: 1459172.00	0 (0.00%)	635
Maintenance Labor	Ratio	Min: -496.98 Max: 338080.51	14 (0.30%)	475
Aircraft and Traffic Handling Personnel	Ratio	Min: -4488.69 Max: 524905.00	61 (1.30%)	829

Other Personnel	Ratio	Min: -3640.49 Max: 795532.00	155 (3.31%)	751
Total Salaries	Ratio	Min: -1594.98 Max: 2522590.00	0 (0.00%)	609
Personnel Expense	Ratio	Min: -84.43 Max: 204970.00	62 (1.33%)	523
Employee Benefits and Pensions	Ratio	Min: -2750.83 Max: 710188.00	0 (0.00%)	668
Payroll Taxes	Ratio	Min: -29341.00 Max: 180016.00	0 (0.00%)	663
Total Related Fringe Benefits	Ratio	Min: -1750.63 Max: 1037103.00	0 (0.00%)	593
Total Salaries and Related Fringe Benefits	Ratio	Min: -1891.11 Max: 3496667.00	0 (0.00%)	594
Aircraft Fuel and Oil	Ratio	Min: -424.67 Max: 2295238.00	3 (0.06%)	668
Maintenance Material	Ratio	Min: -6656.00 Max: 235536.00	3 (0.06%)	528
Passenger Food	Ratio	Min: -1532.84 Max: 288919.00	382 (8.17%)	873
Other Materials	Ratio	Min: -4312.86 Max: 125117.00	141 (3.02%)	641
Total Materials	Ratio	Min: -418.59 Max: 2634359.00	0 (0.00%)	660
Advertising and Other Promotions	Ratio	Min: -7645.41 Max: 118797.00	60 (1.28%)	745
Communication	Ratio	Min: -21286.01 Max: 99656.69	0 (0.00%)	784
Insurance	Ratio	Min: -16551.00 Max: 49232.00	0 (0.00%)	533
Outside Flight Equipment Maintenance	Ratio	Min: -4353.50 Max: 421019.01	48 (1.03%)	557
Traffic Commissions - Passenger	Ratio	Min: -2593.00 Max: 127257.19	536 (11.46%)	863
Traffic Commissions - Cargo	Ratio	Min: -1068.62 Max: 13369.97	343 (7.34%)	808
Other Services	Ratio	Min: -50930.16 Max: 1035782.00	0 (0.00%)	772
Total Services	Ratio	Min: -46388.55 Max: 1622251.00	0 (0.00%)	587
Landing Fees	Ratio	Min: -855.40 Max: 269000.76	3 (0.06%)	674

Rentals	Ratio	Min: -1209.95 Max: 600969.36	3 (0.06%)	555
Depreciation	Ratio	Min: -998.40 Max: 440106.00	0 (0.00%)	699
Amortization	Ratio	Min: -800.12 Max: 242875.81	0 (0.00%)	835
Other Expenses	Ratio	Min: -165100.00 Max: 1373594.48	97 (2.07%)	665
Transport Related Expense	Ratio	Min: -7589.00 Max: 11751294.00	124 (2.65%)	886
Total Operating Expenses	Ratio	Min: -3053.92 Max: 13654442.00	1 (0.02%)	683
Carrier Name	Nominal	84 unique categories (e.g., Florida West Airlines Inc., USA Jet Airlines Inc., Lynden Air Cargo Airlines...)	0 (0.00%)	N/A
UNIQUE_CARRIER_ENTITY	Nominal	137 unique categories (e.g., 06747, 16917, 11040, ...)	0 (0.00%)	N/A
Carrier's Operation Region	Nominal	Domestic, International, Latin America, Atlantic, Pacific, System	0 (0.00%)	N/A
CARRIER_GROUP_NEW	Ordinal	Values: 1, 2, 3	0 (0.00%)	None
CARRIER_GROUP	Ordinal	Values: 1, 2, 3	0 (0.00%)	None
Year	Interval	Min: 2014 Max: 2025	0 (0.00%)	None
Quarter	Interval	Min: 1 Max: 4	0 (0.00%)	None

* All Ratio values are measured in \$Thousands.

** Outliers are defined as values beyond 1.5 times the Interquartile range (IQR) (Tukey, 1977).

9. Missing Attribute Values: Refer to #8

10. Outliers using IQR method: Refer to #8

11. Class Distribution: Number of instances per year by quarter for Delta Airlines

Class (Year)	N (Percent)
2014	16 (8.51%)
2015	16 (8.51%)
2016	16 (8.51%)
2017	16 (8.51%)
2018	16 (8.51%)

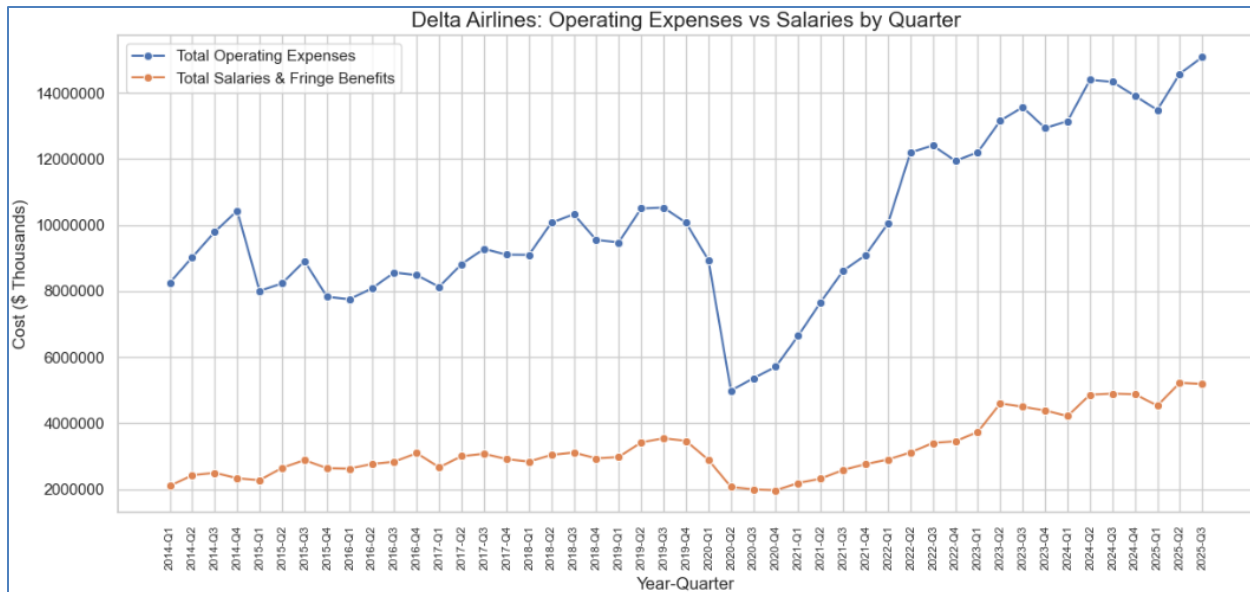
2019	16 (8.51%)
2020	16 (8.51%)
2021	16 (8.51%)
2022	16 (8.51%)
2023	16 (8.51%)
2024	16 (8.51%)
2025	12 (6.38%) *

* Incomplete BTS Form-41 data for Q4'2025

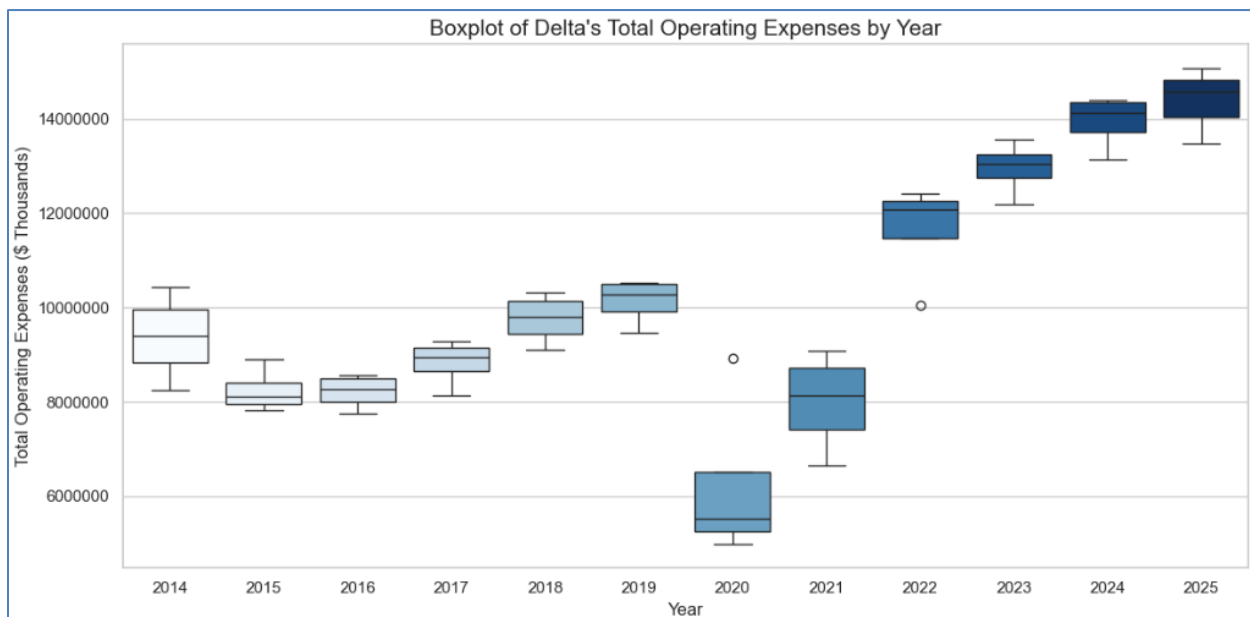
12. Analysis:

A descriptive statistical analysis approach for Delta Airlines is provided below.

- Summary Plot of Total Expenses and Total Salary & Fringe Benefits by Year/Quarter



- Total Operating Expenses by Year

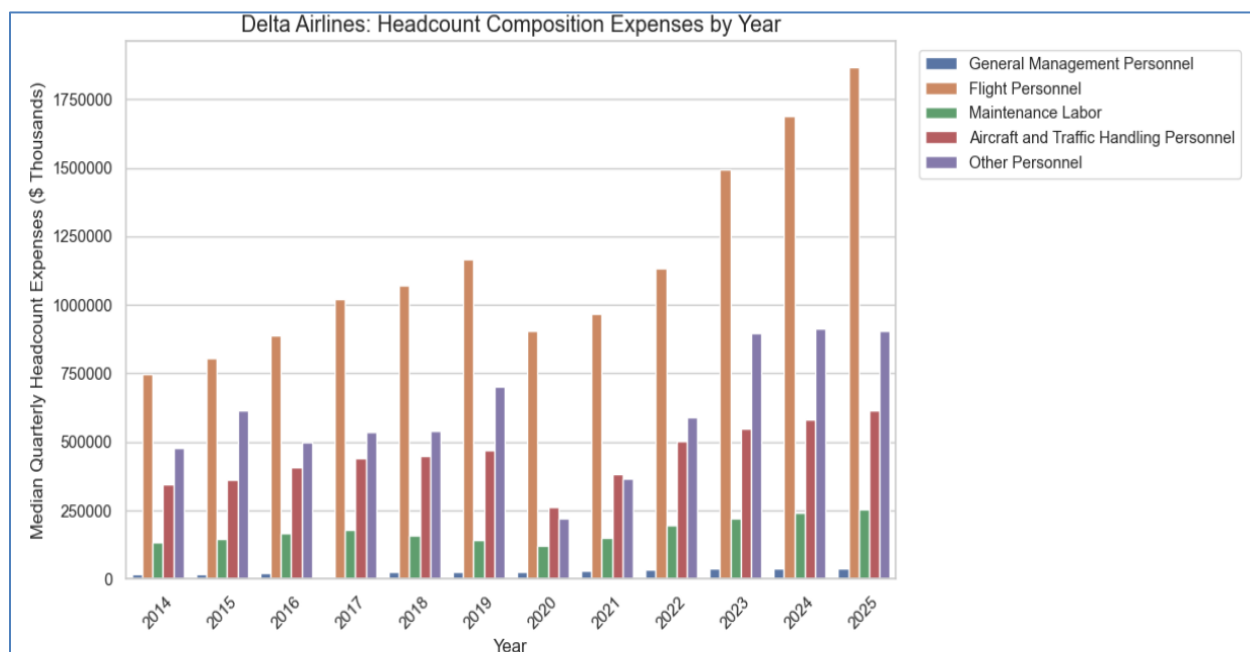


Year	Mean	Median	Min	Max	Std Dev	Count	IQR
2014	9375486.50	9407509.0	8255620.0	10431308.0	942470.20	4	1118873.50
2015	8242778.50	8117548.0	7827924.0	8908094.0	473382.68	4	441591.50
2016	8218295.50	8280487.0	7751341.0	8560867.0	376014.79	4	504295.50
2017	8831532.00	8960013.0	8128162.0	9277940.0	505629.40	4	499468.00
2018	9758523.25	9807009.0	9095452.0	10324623.0	547813.68	4	697675.25

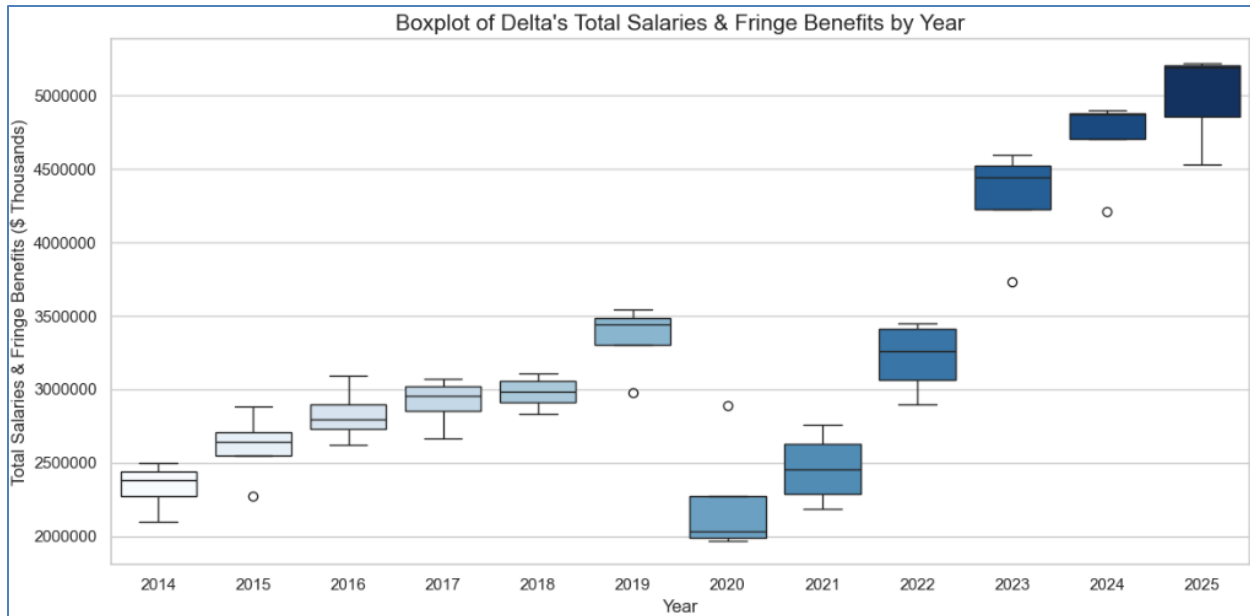
2019	10141886.50	10283964.5	9471976.0	10527641.0	493934.98	4	589033.00
2020	6247578.75	5533952.5	4989625.0	8932785.0	1814190.70	4	1251079.25
2021	8002881.50	8137737.0	6647571.0	9088481.0	1080554.27	4	1322552.00
2022	11647755.00	12067665.5	10049572.0	12406117.0	1082329.10	4	779929.50
2023	12963421.00	13043711.5	12199620.0	13566641.0	572914.49	4	507888.50
2024	13944599.50	14120304.5	13142154.0	14395635.0	577181.17	4	632130.00
2025	14379549.33	14576607.0	13483141.0	15078900.0	815926.15	3	797879.50

Delta's total operating expenses dropped sharply in 2020, with median quarterly costs reaching their lowest level at approximately \$5.5B, reflecting reduced flight activity during COVID-19. Expenses then rose steadily during recovery, increasing from about \$8.1B in 2021 to roughly \$14.1B in 2024, with partial 2025 data suggesting expenses will likely exceed prior years. Elevated IQR values in 2014, 2020, and 2021 indicate greater quarterly volatility. Two notable outliers appear in the boxplot; an unusually high quarterly expense in early 2020, and an unusually low quarterly expense in early 2022. These values are outliers compared to other quarters within their respective years and suggest temporary operational adjustments rather than sustained trends.

- Headcount expenses composition by Year



- Total Salary and Fringe Benefit Expenses by Year



Year	Mean	Median	Min	Max	Std Dev	Count	IQR
2014	2340198.50	2380304.5	2102261.0	2497924.0	172095.53	4	167178.50
2015	2611347.50	2642999.5	2275254.0	2884137.0	251288.68	4	160528.50
2016	2828995.50	2799672.5	2623213.0	3093424.0	196930.17	4	169035.00
2017	2913732.50	2956873.0	2665510.0	3075674.0	178285.31	4	167884.00
2018	2980851.00	2987740.0	2835827.0	3112097.0	120743.98	4	146901.00
2019	3351380.00	3442448.5	2975580.0	3545043.0	255883.24	4	177715.50
2020	2231797.00	2034269.5	1970724.0	2887925.0	439535.24	4	286185.50
2021	2463861.50	2453153.0	2189278.0	2759862.0	256711.73	4	339543.50
2022	3217851.50	3258496.5	2901346.0	3453067.0	257985.09	4	352338.00
2023	4303685.25	4441941.0	3734694.0	4596165.0	388929.15	4	299373.75
2024	4713242.00	4870962.5	4213819.0	4897224.0	333222.60	4	180783.50
2025	4980851.00	5188572.0	4532178.0	5221803.0	388917.31	3	344812.50

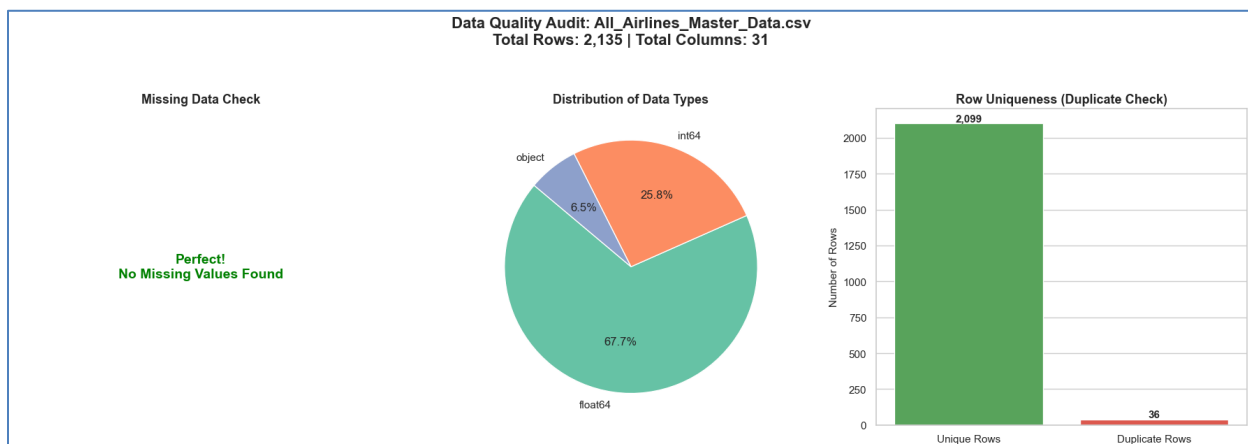
Delta's salary and fringe benefit expenses declined sharply in 2020, with median quarterly costs falling to approximately \$2.03B. Salary expenses then rose steadily, increasing from about \$2.45B in 2021 to roughly \$4.87B in 2024, with partial 2025 data showing a median of \$5.19B, indicating continued growth in labor costs. Elevated IQR values in 2020 (\$0.29B), 2021 (\$0.34B), and 2022 (\$0.35B) indicate greater quarterly volatility during the recovery period. Several notable outliers appear in the boxplot including an unusually high quarterly salary expense in 2020 and an unusually low quarter in 2023. In addition to low quarters in 2015, 2019 and 2024 these appear to highlight temporary staffing adjustments rather than sustained trends.

INVESTIGATING FOR ISSUES

Addressing issues (errors and inconsistencies) and the reporting cadence gaps that need to be addressed within the three data sources: Flight Statistics, Employee Data, and Financial Data.

Flight Statistics

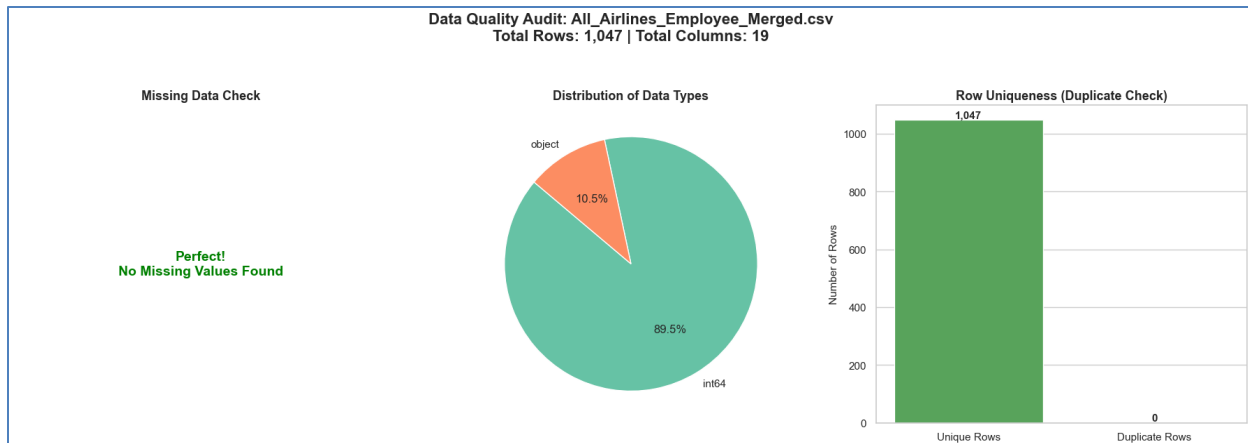
- Inconsistent Source File Naming
 - Issue: Varying month lengths (28, 30, 31 days) in file name making manual aggregation prone to error
 - Solution: Implement script to normalize file names into a YYYY_MM format
- Structural Inconsistency
 - Issue: Monthly source files are currently unstructured
 - Solution: Implement a scraping and looping script to transpose raw data into a consolidated master dataset
- Unstructured Metadata
 - Issue: Source files contain header text in an unstructured format
 - Solution: Use string manipulation to scrape header lines as distinct variables
- Missing Data
 - Issue: December 2025 records are currently missing
 - Solution: Populate when data is made available.
- Data Type Inconsistency
 - Issue: Date is stored as a long-form string
 - Solution: Apply data type conversion to parse the string into a standardized date format then extract numeric Year and Month integers
- Duplicate Rows
 - Issue: 36 duplicate rows that need to be addressed
 - Solution: Validating the source files that were extracted to identify the point of duplication. Duplicates to be removed.



Employee Data

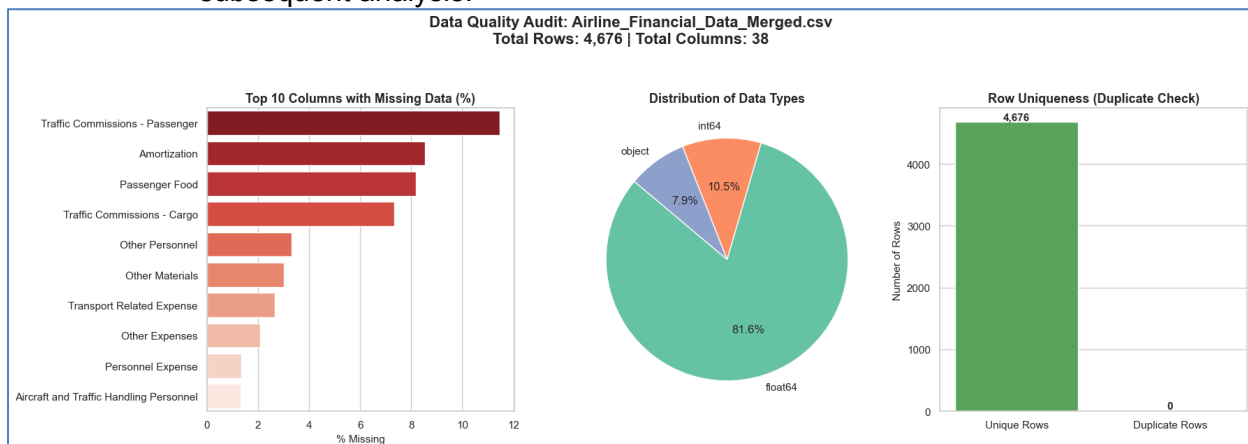
- Split Information
 - Issue: Employee information for Delta Airlines Inc. is split between 4 different operating regions (Atlantic, Domestic, Latin America, Pacific).
 - Solution: Summate the Delta information for each region per year. Data for each region may still be utilized.
- Granularity Inconsistency
 - Issue: The data available for Delta Airlines Inc. is given as yearly values, while

- the other datasets are given as monthly and quarterly values.
- Solution: Addressed below in **Mismatched Reporting Cadence**



Financial Data

- Domain Format Error
 - Issue: UNIQUE_CARRIER_ENTITY column contains mix of string values (e.g. "06841") and numeric values (e.g. 16917) and thus violating its expected domain format. This feature should be cast as a string, since this is categorical data.
 - Solution: Apply data type conversion during ingestion to cast this column as a string.
- Missing Values
 - Multiple missing values within the raw financial dataset (that contains every Carrier) for various features (see Missing Data horizontal barplot below).
 - Solution: Since the intent of the resultant analysis is to evaluate Delta Air Lines specifically, the datasets will first be filtered to Delta-relevant records prior cleaning and analysis. Any missing data flagged post-filtering will be investigated contextually and imputed with results using a documented external data source.
- Missing Tuples
 - Issue: The most recent year in the dataset is 2025, containing Q1–Q3 and missing Q4 indicating partial reporting.
 - Solution: Document the temporal coverage window and populate when data is made available, if possible. If not possible, then exclude this timeframe from any subsequent analysis.



Mismatched Reporting Cadence

Issue: The three data sources report at different frequencies – Flight (Monthly), Financial (Quarterly), and Employee (Yearly) – preventing a direct 1:1 merge.

Solutions:

- Yearly Consolidation: Aggregate all data to Yearly
 - Trade-off: Simplest to execute but results in significant data loss
- Monthly Estimation: Stretch yearly and quarterly data into monthly estimates by averaging them out.
 - Trade-off: Imputes data to not the truest form aka “fake data”
- Smooth-out Method (MIDAS): Regress a low-frequency variable (yearly) directly on high-frequency variables (monthly lags). Uses a polynomial weighting function to determine how much the recent monthly data impacts the yearly outcome.
 - Trade-off: Adds complexity to the modeling process

Outliers

Issue: As can be seen in the Collect Data section, nearly every ratio-scale variable across all three datasets contains numerous statistical outliers (via IQR method) when all carriers are evaluated together with the exception of **All_Average Airborne Time** in the Flight Statistics dataset. However, this is expected behavior rather than true anomalies since these datasets contain carriers ranging from small regional operators (e.g., Horizon Air) to large-scale enterprises (e.g., United Parcel Service, Delta Air Lines), producing naturally heavy-tailed distributions.

Solution: Since the intent of the resultant analysis is to evaluate Delta Air Lines specifically, the impact of outlier influence on analysis will only be evaluated on Delta-relevant records, where remaining outliers are more likely to represent genuine anomalies, such as quarters with atypical operational disruptions, rather than artifacts of cross-carrier scale differences. Any outliers flagged post-filtering will be investigated contextually rather than removed automatically, consistent with the project's objectives. For example, since we want to evaluate the impact of seasonal changes, then a quarter showing unusually high cancellation rates or operating expenses may reflect a genuine peak-season disruption and removing it would eliminate the very signal the analysis is designed to detect.

CLEANING DATA

Python was used to extract data from the source websites, then scraped and cleaned. Snapshots of scripts are included to illustrate the data cleaning steps implemented to address the issues outlined above. For a complete view of the process, please refer to the full script provided in the Appendix.

Flight Statistics

- Inconsistent Source File Naming

```
# --- RENAME FILE ---
files = [f for f in os.listdir(DOWNLOAD_DIR) if f.endswith(".csv")]
if files:
    full_paths = [os.path.join(DOWNLOAD_DIR, f) for f in files]
    newest_file = max(full_paths, key=os.path.getctime)

    # Target Name: Delta_2015_01.csv
    new_name = os.path.join(DOWNLOAD_DIR, f"Delta_{year}_{month:02d}.csv")

    if newest_file != new_name:
        if os.path.exists(new_name):
            os.remove(new_name)
        shutil.move(newest_file, new_name)
    print(f"    -> Saved: Delta_{year}_{month:02d}.csv")
```

- Removes dependency on day ranges and normalizes file names into a YYYY_MM format

- Structural Inconsistency

```
# --- EXTRACT TABLE 1: ALL FLIGHTS ---
df_all = pd.read_csv(file, skiprows=4, nrows=1)
df_all.columns = [f"All_{col.strip()}" for col in df_all.columns]

# --- EXTRACT TABLE 2: LATE FLIGHTS ---
df_late = pd.read_csv(file, skiprows=8, nrows=1)
df_late.columns = [f"Late_{col.strip()}" for col in df_late.columns]

# --- EXTRACT TABLE 3: CAUSE OF DELAY ---
df_cause = pd.read_csv(file, skiprows=12, nrows=1)
df_cause.columns = [f"Cause_{col.strip()}" for col in df_cause.columns]

# --- COMBINE TABLES ---
combined_row = pd.concat([
    df_all.reset_index(drop=True),
    df_late.reset_index(drop=True),
    df_cause.reset_index(drop=True)
], axis=1)
```

- Extracts slices of data into a structured format and converts to standardized rows per month to consolidate into a master dataset

- Unstructured Metadata

```
# --- ROBUST METADATA EXTRACTION ---
with open(file, 'r', encoding='utf-8') as f:
    lines = [next(f) for _ in range(3)]

airline_line = lines[1].strip() # Example: "Airline: American Airlines Inc. (AA)"
time_line = lines[2].strip() # Example: "Time Period: Thursday, January 1, 2015 to..."

# 1. Extract BOTH Airline Name and Airline Code
# This regex captures everything after "Airline: " up to the "(" as the Name,
# and everything inside the "(" as the Code.
match = re.search(r'Airline:\s*(.*?)\s*\((.*?)\)$', airline_line)
if match:
    airline_name = match.group(1).strip() # "American Airlines Inc."
    airline_code = match.group(2).strip() # "AA"
else:
    airline_name = "UNKNOWN"
    airline_code = "UNKNOWN"
```

- Converts free-text headers into distinct variables using string manipulation

- Missing Data

```

try:
    driver.find_element(By.ID, "DL_CSV").click()
    time.sleep(2) # Wait for download

    # --- RENAME FILE ---
    files = [f for f in os.listdir(DOWNLOAD_DIR) if f.endswith(".csv")]
    if files:
        full_paths = [os.path.join(DOWNLOAD_DIR, f) for f in files]
        newest_file = max(full_paths, key=os.path.getctime)

        # Target Name: AA_2015_01.csv, UA_2015_01.csv, etc.
        new_name = os.path.join(DOWNLOAD_DIR, f"{airline_code}_{year}_{month:02d}.csv")

        if newest_file != new_name:
            if os.path.exists(new_name):
                os.remove(new_name)
            shutil.move(newest_file, new_name)
            print(f"    [Saved]: {airline_code}_{year}_{month:02d}.csv")

except Exception as dl_error:
    print(f"    [Skip]: No download link (Data missing for this month).")

```

- Rerun script to account for missing months when source has it populated
- Data Type Inconsistency

```

# 2. Extract Year and Month from the "Time Period" text
date_part = time_line.split(" to ")[0].replace("Time Period: ", "").strip()
parsed_date = datetime.strptime(date_part, "%A, %B %d, %Y")
year = parsed_date.year
month = parsed_date.month

```

- Apply data type conversion to parse the string into a standardized date format then extract numeric Year and Month integers
- Duplicate Rows

```

# 2. CREATE MASTER DATAFRAME
if all_monthly_data:
    master_df = pd.concat(all_monthly_data, ignore_index=True)

    master_df.drop_duplicates(inplace=True)

# 3. SORT BY AIRLINE NAME, THEN DATE
master_df.sort_values(by=['Airline_Name', 'Year', 'Month'], inplace=True)

# 4. SAVE TO CSV
master_df.to_csv(output_file, index=False)
print(f"\nSuccess! Merged {len(master_df)} monthly records into '{output_file}'.")
print(master_df.head()) # Show first few rows

```

- Removed duplicate rows before exporting to CSV

Employee Data

- Total Employees Calculation Error

```

employee_cols = df.columns[3:-1] # select only employee columns

subset = df[
    ((df['Carrier Name'] == 'Northern Air Cargo Inc.') & (df['Year'] == 2014)) |
    ((df['Carrier Name'] == 'Envoy Air') & (df['Year'] == 2023)) |
    ((df['Carrier Name'] == 'PSA Airlines Inc.') & (df['Year'] == 2023)) |
    ((df['Carrier Name'] == 'United Parcel Service') & (df['Year'] == 2023))
].copy()

# check difference between Total Employees column and sum of employee columns
subset['Calculated Total BEFORE'] = subset[employee_cols].sum(axis=1)
subset['Difference BEFORE'] = subset['Calculated Total BEFORE'] - subset['Total Employees']

print(subset[['Difference BEFORE']])

# fix calculations
df.loc[
    (
        ((df['Carrier Name'] == 'Northern Air Cargo Inc.') & (df['Year'] == 2014)) |
        ((df['Carrier Name'] == 'Envoy Air') & (df['Year'] == 2023)) |
        ((df['Carrier Name'] == 'PSA Airlines Inc.') & (df['Year'] == 2023)) |
        ((df['Carrier Name'] == 'United Parcel Service') & (df['Year'] == 2023))
    ) & (df[employee_cols].sum(axis=1) != df['Total Employees']), 'Total Employees'] = df[employee_cols].sum(axis=1)

# check difference after fix
subset_after = df[
    ((df['Carrier Name'] == 'Northern Air Cargo Inc.') & (df['Year'] == 2014)) |
    ((df['Carrier Name'] == 'Envoy Air') & (df['Year'] == 2023)) |
    ((df['Carrier Name'] == 'PSA Airlines Inc.') & (df['Year'] == 2023)) |
    ((df['Carrier Name'] == 'United Parcel Service') & (df['Year'] == 2023))
].copy()

subset_after['Calculated Total AFTER'] = subset_after[employee_cols].sum(axis=1)
subset_after['Difference AFTER'] = subset_after['Calculated Total AFTER'] - subset_after['Total Employees']

print(subset_after[['Difference AFTER']])

```

- Checks the difference between original total employee column and the sum of the employee columns
- Calculates Total Employees by summing the employee columns to ensure that totals are correct and replaces incorrect values in the Total Employee column

Financial Data

- Domain Format Error

```

for file in files:
    try:
        # Read the CSV file
        #Cast UNIQUE_CARRIER_ENTITY as string to fix domain format
        df = pd.read_csv(file, dtype={'UNIQUE_CARRIER_ENTITY': str})

        # Append all rows directly (No Delta filter)
        if not df.empty:
            all_data.append(df)
            print(f" -> Processed: {os.path.basename(file)} ({len(df)} rows)")

```

- Casts the UNIQUE_CARRIER_ENTITY column as a string during CSV ingestion via the dtype parameter, ensuring categorical carrier entity codes (e.g. "06841") are not misinterpreted as numeric values

- Missing Values

```

if all_data:
    merged_df = pd.concat(all_data, ignore_index=True)

    # 5. Filter to Delta relevant records prior to further cleaning
    merged_df = merged_df[merged_df['CARRIER_NAME'] == 'Delta Air Lines Inc.']

    # 6. Check for missing values post-filtering
    missing = merged_df.isnull().sum()
    missing = missing[missing > 0]
    if not missing.empty:
        print("Missing values after Delta filter:")
        print(missing)

```

- Filters the concatenated dataset to Delta Air Lines records prior to cleaning and analysis, reducing the scope to only Delta-relevant rows
- Checks for any remaining missing values post-filtering; any flagged missing data will be investigated contextually and imputed using a documented external data source
- Missing Tuples

```
# 10. Document Temporal coverage window
year_quarter = merged_df.groupby('Year')['Quarter'].nunique()
incomplete = year_quarter[year_quarter < 4]
if not incomplete.empty:
    print('Incomplete year(s) detected:')
    for yr, qcount in incomplete.items():
        print(f' {yr}: {qcount} of 4 quarters reported')
    print('Note: Exclude or document partial years',
          'before subsequent analysis.')
```

- Identifies incomplete reporting years by comparing the number of unique quarters per year against the expected four
- Documents the temporal coverage window (2025 contains Q1- Q3 only); partial year to be populated when data becomes available, or excluded from subsequent analysis if not

Mismatched Reporting Cadence

The three data sources report at different frequencies – Flight (Monthly), Financial (Quarterly), and Employee (Yearly) – preventing a direct 1:1 merge.

- The data has been cleaned and standardized to ensure it is structured and consistent.
- Solutions to address reporting cadence should be applied at transformation.

Outliers

Outliers will not be removed during data cleaning stage. Instead, they will be noted and handled later during the transformation stage using appropriate methods if needed. This approach helps preserve patterns in the data, such as seasonal or operational changes, rather than removing potentially meaningful values.

Outlier detection was re-applied on each Delta-filtered dataset using the IQR method (Tukey, 1977). Each dataset was evaluated at its native reporting cadence: monthly, quarterly, and yearly prior to any transformation. (see Appendix: Delta-Filtered Outlier Summary (IQR Method) for results output)

```
def detect_outliers_iqr(df, dataset_name):
    """
    Re-applies Tukey IQR-based outlier detection
    on Delta-filtered records. Flags and documents outliers
    rather than removing them.
    """
    numeric_cols = df.select_dtypes(include='number').columns
    results = []

    for col in numeric_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outlier_mask = (df[col] < lower_bound) | (df[col] > upper_bound)
        n_outliers = outlier_mask.sum()

        if n_outliers > 0:
            results.append({
                'Dataset': dataset_name,
                'Feature': col,
                'N_Outliers': n_outliers,
                'Lower_Bound': round(lower_bound, 2),
                'Upper_Bound': round(upper_bound, 2),
                'Min_Value': round(df[col].min(), 2),
                'Max_Value': round(df[col].max(), 2)
            })

    return results
```

- Re-applies IQR outlier detection on **Delta-only** records across all three datasets to separate genuine anomalies from cross-carrier scale artifacts.
- Flags and documents outliers rather than removing them, since we want to investigate spikes that reflect real operational events.
- Any further outlier treatment will be evaluated at the transformation stage.

ANALYSIS

The project scope, consistent with the Data Collection and Cleaning focus, covered sourcing three independently reported Delta Air Lines data marts from BTS, profiling and cleaning each at its native cadence, and transforming them into a single integrated dataset suitable for downstream analysis. This section describes the transformation and integration work that was completed thus summarizing what the resulting dataset supports and explains why the four research queries defined in the Goal section were not fully answered within the time and scope of this project.

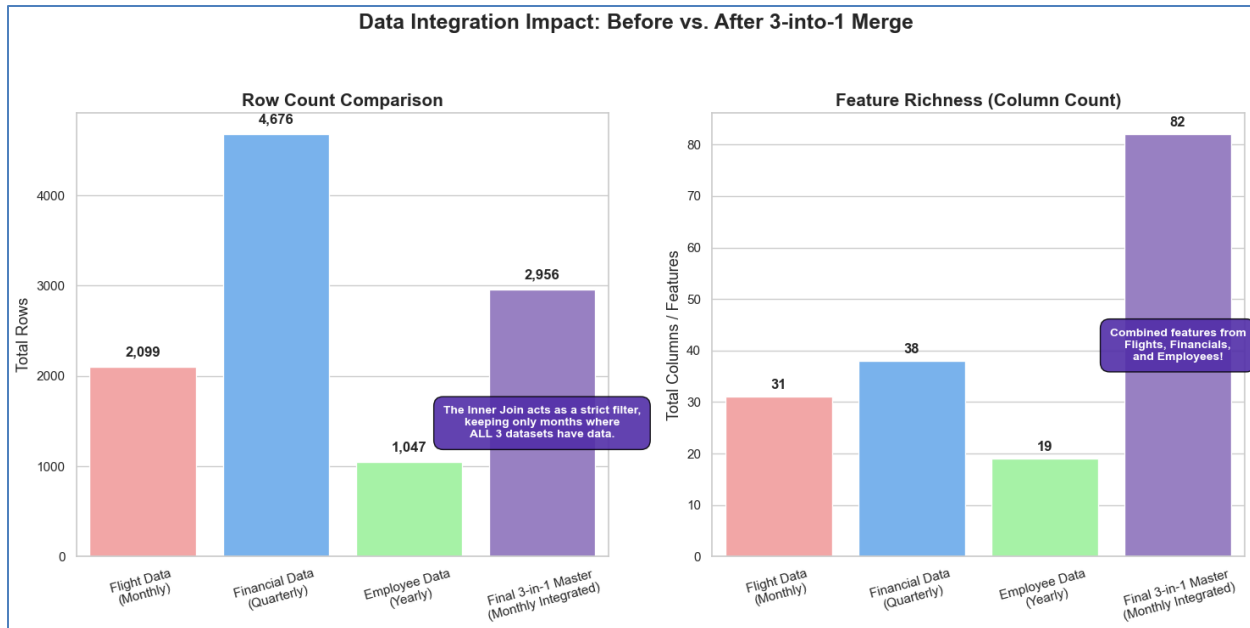
Data Transformation and Integration

After cleaning the three data marts, we confronted the core integration challenge identified earlier in the Investigating for Issues section: the sources were reported at different cadences. Flight Statistics were monthly, Financial Data was quarterly, and Employee Data was yearly (and split across four operating regions). Because the business goal was to measure how seasonal workforce levels impacted peak-season flight cancellations and financial performance, collapsing everything to a yearly cadence would effectively erase the very signal the project is designed to detect. Conversely, trying to disaggregate monthly detail we do not have would fabricate variation.

To solve this, we used a Monthly Estimation method. This allowed us to "stretch" the quarterly and yearly data so that Flight, Financial, and Employee records could be joined on a common (Carrier Name, Year, Month) key. This method preserved totals at the native reporting cadence but, by design, flattened the within-period variation for the stretched sources.

1. **Flight Data.** Already in monthly format. The only transformation was dropping incomplete 2025 records (Q4 2025 is not yet published in BTS Form 41 at time of writing) so they do not distort historical trend comparisons.
2. **Financial Data.** Quarterly in the source. To project to monthly, each quarterly value was divided by three and assigned to each of the three months in that quarter. This preserves quarterly totals by construction and produces an even monthly estimate.
3. **Employee Data.** Yearly in the source and split across four operating regions (Atlantic, Domestic, Latin America, Pacific). Regional subtotals for each Carrier-Year were first summed into a single total headcount. The yearly total was then copied (not divided) into each of the twelve months for that year.

Once all three datasets shared the same (Carrier Name, Year, Month) key, we merged them using an inner join executed in two steps: first Flight with Financial, then the combined table with Employee. The inner join acted as a strict quality filter where any month for which a carrier is missing data in any one source is dropped entirely. This guaranteed no blank fields in the integrated dataset. The resulting file contained one row per carrier per month and is the foundation on which any subsequent analysis would be built.



Data Integration Impact (Before vs. After Merging).

With the transformation and integration complete, the resulting monthly panel was formatted to support the following downstream activities:

- Time-series description of Delta's flight operations, workforce, and cost structure on a consistent monthly key.
- Seasonal comparison of cancellation counts, delay metrics, and related operational measures, since Flight Statistics retained their native monthly variation.
- Year-over-year comparison of workforce levels and cost structure, since the transformations preserved the source totals at their native cadence.

Source	Native Cadence	Transformation	Effect
Flight Statistics	Monthly	Drop incomplete 2025 records; parse dates; deduplicate	No information loss; monthly variation preserved
Financial Data	Quarterly	Divide each quarterly total by 2 and replicate across months	Quarterly totals preserved; within-quarter variation is flattened
Employee Data	Yearly	Sum regions to annual total; copy annual total to each month	Annual headcount preserved; within-year seasonality is flattened

Summary of Transformations applied to address different cadences of the three data marts.

Descriptive summaries produced earlier represent the full extent of analytical work completed. No inferential modeling, formal hypothesis testing, or predictive analysis was performed, and none is claimed.

Why the Research Queries Were Not Fully Addressed

The project's four research queries, restated below, were scoped to frame the downstream analytical agenda rather than to be answered within the data collection and cleaning phase. The sections that follow explain, for each query, specifically why it was not addressed in this project and what the data collected so far can and cannot yet say about it.

Query 1: What is Delta's flight cancellation rate during peak travel seasons?

The Flight Statistics data mart contained the raw inputs needed to compute monthly cancellation counts and cancellation rates for Delta across 2015-2025, and the descriptive summaries in the Collect Data section showed that cancellation volume varies materially across the study window. However, producing a defensible "peak-season cancellation rate" figure requires more than the descriptive profiling completed so far: it requires defining peak-season windows formally (calendar-based or traffic-based), separating the 2020 pandemic year as a structural break, and comparing across years in a way that controls for underlying volume. Those steps were not performed in this submission because they fell outside the scope of data collection and cleaning.

Query 2: How does employment headcount and employment level change over time? Are there seasonal changes?

The sub-question about level change over time is partially supported by the cleaned data; Delta's total annual headcount across the four regions is available for 2014-2024 and was profiled in the Employee Data section. However, the sub-questions about employment level (full-time versus part-time) and seasonal changes could not be addressed with the data that was collected. The BTS employment table pulled for this project reports only yearly totals; the monthly employment schedule, which contained the within-year variation needed to examine seasonality and the FT/PT breakdown, was a separate source that was not in scope for this phase of the project.

Query 3: Can we utilize employee headcount to predict flight cancellations or vice versa?

A predictive relationship between headcount and cancellations requires inferential modeling, which was not performed. Beyond that scope consideration, the current dataset had a structural limitation that would need to be addressed before any month-level predictive work could be credible: because yearly employee totals are copied into each month of the year, the headcount variable has no within-year variation. Any model fit on monthly data with the current dataset would rely entirely on cancellation variation; the headcount variable would effectively function as an annual-level control.

Query 4: Can we utilize employee headcount to predict financial performance or vice versa?

The Financial Data mart collected for this project covers the expense side of Delta's operations: salaries, fringe benefits, fuel, maintenance, services, and other operating-expense line items. It does not contain revenue, capacity (available seat miles), or additional economics measures such as Revenue per Available Seat Mile (RASM) which are the variables typically meant by "financial performance" in an airline context. The predictive relationship between headcount and cost is supported by the dataset but would, once again, require inferential modeling that was not performed. A fuller answer to the query as written, linking headcount to profitability rather than cost, would require additional data collection of revenue and capacity tables from Form 41 or from Delta's investor relations filings.

CONCLUSION

This project produced a cleaned, integrated monthly dataset for Delta Air Lines that brought together three independently reported BTS data marts onto a single (Carrier, Year, Month) key.

These three independent data marts (Flight Statistics, Employee Data, and Financial Data) were sourced directly from authoritative BTS data, independently profiled, and reconciled end-to-end onto the shared (Carrier, Year, Month) key. Data quality issues encountered along the way, including inconsistent file naming, structural inconsistency, unstructured metadata, missing data, duplicate rows, split regional information, domain-format errors, and missing tuples, were each identified, recorded, and addressed with a documented solution, providing full traceability from raw source to cleaned output. Key methodological choices were made deliberately rather than by default and the result was an integrated dataset ready for downstream analysis, with additional external variables easily joined in on the existing key without further restructuring.

The most important limitation of this project was that the work was descriptive rather than inferential: profiling, cleaning, and transformation were performed, but statistical modeling, hypothesis testing, and predictive analysis were not, which meant the four research queries remained unanswered and were scoped as the downstream agenda the cleaned dataset would enable. Temporal coverage was also incomplete for 2025, since Q4 Form 41 data had not yet been published and December Flight Statistics records were unavailable at extraction time, so 2025 should be excluded from historical trend comparisons. Finally, no external control variables were present in the dataset: drivers of cancellations and financial performance that originated outside Delta's workforce (e.g. weather, fuel prices, macro demand, and labor actions) were out of scope for the data collection phase and are discussed as potential future-work considerations below.

The full research agenda would require work beyond the scope of this project in two distinct categories: additional data to collect and integrate, and analytical methods to apply to the combined dataset. The integrated dataset produced by this project was designed from the outset to accept such additions on its existing (Year, Month) key, so no structural rework would be needed to extend it.

On the data side, two coverage gaps would need to be closed before the original queries could be answered directly. Monthly employment data from BTS, including the FT/PT breakdown, would address the seasonality sub-question in Query 2 and replace flat annual headcount for Query 3. Revenue and capacity measures from BTS Form 41 and Delta's investor filings would allow Query 4 to be framed in terms of profitability rather than cost. Beyond those gaps, external controls would need to be merged in to guard against omitted-variable concerns, such as weather severity, jet fuel prices, macro demand, labor actions, airport capacity, and fleet composition.

On the methods side, the expanded dataset would support formal seasonality analysis for Query 1, regression or panel modeling with controls for Query 3, parallel cost and profitability modeling for Query 4, event-window analysis to isolate one-off shocks from seasonal patterns, and explicit handling of the 2020 pandemic year as a structural break.

In summary, this project delivered a clean, well-documented Delta Air Lines dataset that brings three distinct sources together into a single, analysis-ready foundation, positioning the team to revisit the original research questions as the scope of work expands.

REFERENCES

- Delta Air Lines, Inc. (n.d.). *Financials*. Delta Air Lines Investor Relations. <https://ir.delta.com/financials/default.aspx#financial-table>
- Diersen, V., Koppelman, R., Lauver, N., Naik, A., & Vorderstrasse, D. (2024). *Delta Airlines Strategic Audit* (No. 703) [Undergraduate honors thesis, University of Nebraska-Lincoln]. DigitalCommons@University of Nebraska - Lincoln. <https://digitalcommons.unl.edu/honorstheses/703/>
- Müller, H., & Freytag, J.-C. (2003). Problems, methods, and challenges in comprehensive data cleansing (Technical Report HUB-IB-164). Humboldt-Universität zu Berlin.
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesely.
- U.S. Department of Transportation, Bureau of Transportation Statistics. (n.d.). *Air Carrier financial: Schedule P-10.OST_R | BTS*. TranStats. https://www.transtats.bts.gov/DL_SelectFields.aspx?gnoyr_VQ=GDF&QO_fu146_anzr=Nv4%20Pn44vr4%20Sv0n0pvny
- U.S. Department of Transportation, Bureau of Transportation Statistics. (n.d.). *Air carrier financial data (Form 41 financial reports)*. TranStats. https://www.transtats.bts.gov/Tables.asp?QO_VQ=EGI&QO_anzr=Nv4%20Pn44vr4%20Sv0n0pvny%20FDer21465%20FD%20FLS14z%20FDHE%20Sv0n0pvny%20FDQn6n%20FM&QO_fu146_anzr=Nv4%20Pn44vr4%20Sv0n0pvny
- U.S. Department of Transportation, Bureau of Transportation Statistics. (n.d.). *Search Airline Employment Data by Month*. TranStats. <https://www.transtats.bts.gov/employment/>
- U.S. Department of Transportation, Bureau of Transportation Statistics. (n.d.). *Summary Statistics Airline*. TranStats. <https://www.transtats.bts.gov/ontime/Airline.aspx>
- U.S. Government Accountability Office. (2023). *Airline Passenger Protections Observations on Flight Delays and Cancellations, and DOT's Efforts to Address Them* (GAO-23-105524). <https://www.gao.gov/assets/gao-23-105524.pdf>
- U.S. Government Accountability Office. (2023). *Airline Passenger Protections Observations on Flight Delays and Cancellations, and DOT's Efforts to Address Them* (GAO-23-105524). <https://www.gao.gov/assets/gao-23-105524.pdf>
- Kotronoulas, G., Miguel, S., Dowling, M., Fernández-Ortega, P., Colomer-Lahiguera, S., Bağçivan, G., Pape, E., Drury, A., Semple, C., Dieperink, K. B., & Papadopoulou, C. (2023). An overview of the fundamentals of data management, analysis, and interpretation in quantitative research. *Seminars in Oncology Nursing*, 39(2), Article 151398. <https://doi.org/10.1016/j.soncn.2023.151398>

APPENDIX A

The full Python script is provided below for reference. It outlines the end-to-end process used for data extraction, cleaning, and transformation for all three data marts.

Code

```
#####  
# Scraping Statistics Data  
#####  
  
%pip install selenium  
%pip install webdriver-manager  
  
import os  
import time  
import calendar  
import shutil  
import pandas as pd  
import numpy as np  
import glob  
import re  
from datetime import datetime  
from selenium import webdriver  
from selenium.webdriver.chrome.service import Service  
from selenium.webdriver.common.by import By  
from selenium.webdriver.support.ui import Select  
from webdriver_manager.chrome import ChromeDriverManager  
  
#####  
# Raw Data Import for all airline statistics  
#####  
  
# --- CONFIGURATION ---  
DOWNLOAD_DIR = os.path.join(os.getcwd(), "Airline_Monthly_Stat") # Changed folder name  
START_YEAR = 2015  
END_YEAR = 2025  
TARGET_URL = "https://www.transtats.bts.gov/ontime/Airline.aspx"  
  
# 1. Create Download Directory  
if not os.path.exists(DOWNLOAD_DIR):  
    os.makedirs(DOWNLOAD_DIR)  
  
# 2. Setup Chrome  
chrome_options = webdriver.ChromeOptions()  
prefs = {"download.default_directory": DOWNLOAD_DIR}  
chrome_options.add_experimental_option("prefs", prefs)  
# chrome_options.add_argument("--headless") # Recommended to uncomment this if running for hours  
  
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=chrome_options)  
  
try:  
    print("Opening Website...")  
    driver.get(TARGET_URL)  
    time.sleep(5) # Give it extra time to load initially  
  
# 3. GET ALL AIRLINES FROM DROPDOWN  
try:  
    airline_dropdown = Select(driver.find_element(By.ID, "cboAirline"))  
    # Create a list of tuples: (Airline_Code, Airline_Name)  
    # We ignore options that have an empty value (like "Select Airline")  
    airlines_to_scrape = [  
        (opt.get_attribute("value"), opt.text)  
        for opt in airline_dropdown.options
```

```

        if opt.get_attribute("value")
    ]
    print(f"Found {len(airlines_to_scrape)} airlines to process.")
except Exception as e:
    print(f"Could not load airline list: {e}")
    driver.quit()
    exit()

# --- LOOP AIRLINES ---
for airline_code, airline_name in airlines_to_scrape:
    print(f"\n=====")
    print(f"Starting: {airline_name} ({airline_code})")
    print(f"=====")

    # Select the current airline
    try:
        Select(driver.find_element(By.ID, "cboAirline")).select_by_value(airline_code)
        time.sleep(2)
    except Exception as e:
        print(f"Could not select {airline_name}. Skipping to next. Error: {e}")
        continue

# --- LOOP YEARS ---
for year in range(START_YEAR, END_YEAR + 1):

    # --- LOOP MONTHS ---
    for month in range(1, 13):
        # Skip future dates
        if year == 2026 and month > 2:
            break

        # Calculate the last day (28, 30, or 31)
        last_day = calendar.monthrange(year, month)[1]
        month_name = calendar.month_name[month]

        print(f" -> Processing: {airline_code} - {month_name} {year}...")

        try:
            # --- STEP 1: SET YEARS FIRST ---
            Select(driver.find_element(By.ID, "stdatyear")).select_by_value(str(year))
            Select(driver.find_element(By.ID, "eddatyear")).select_by_value(str(year))

            # --- STEP 2: SET MONTHS ---
            Select(driver.find_element(By.ID, "stdatemon")).select_by_value(str(month))
            Select(driver.find_element(By.ID, "eddatemon")).select_by_value(str(month))

            time.sleep(1) # Wait for Day dropdown to populate

            # --- STEP 3: SET DAYS ---
            Select(driver.find_element(By.ID, "stdateday")).select_by_value("1")
            Select(driver.find_element(By.ID, "eddateday")).select_by_value(str(last_day))

            # --- STEP 4: SUBMIT ---
            driver.find_element(By.ID, "btnSubmit").click()

            time.sleep(2) # Wait for table to load

            # --- STEP 5: DOWNLOAD CSV ---
            try:
                driver.find_element(By.ID, "DL_CSV").click()
                time.sleep(2) # Wait for download

                # --- RENAME FILE ---
                files = [f for f in os.listdir(DOWNLOAD_DIR) if f.endswith(".csv")]

```

```

if files:
    full_paths = [os.path.join(DOWNLOAD_DIR, f) for f in files]
    newest_file = max(full_paths, key=os.path.getctime)

    # Target Name: AA_2015_01.csv, UA_2015_01.csv, etc.
    new_name = os.path.join(DOWNLOAD_DIR, f"{airline_code}_{year}_{month:02d}.csv")

    if newest_file != new_name:
        if os.path.exists(new_name):
            os.remove(new_name)
        shutil.move(newest_file, new_name)
        print(f"    [Saved]: {airline_code}_{year}_{month:02d}.csv")

except Exception as dl_error:
    print(f"    [Skip]: No download link (Data missing for this month).")

except Exception as e:
    print(f"    [Error] on {month_name} {year}: {e}")
    # If the page crashed, we need to make sure the airline is re-selected for the next loop
    try:
        driver.get(TARGET_URL)
        time.sleep(3)
        Select(driver.find_element(By.ID, "cboAirline")).select_by_value(airline_code)
    except:
        pass
    continue

time.sleep(1)

finally:
    driver.quit()
    print("\nScraping Complete for all airlines.")

#####
# Raw Data Import for Delta Airline
#####

# --- CONFIGURATION ---
DOWNLOAD_DIR = os.path.join(os.getcwd(), "delta_monthly_stats_test")
START_YEAR = 2015
END_YEAR = 2025
TARGET_URL = "https://www.transtats.bts.gov/ontime/Airline.aspx"

# 1. Create Download Directory
if not os.path.exists(DOWNLOAD_DIR):
    os.makedirs(DOWNLOAD_DIR)

# 2. Setup Chrome
chrome_options = webdriver.ChromeOptions()
prefs = {"download.default_directory": DOWNLOAD_DIR}
chrome_options.add_experimental_option("prefs", prefs)
# chrome_options.add_argument("--headless")

driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=chrome_options)

try:
    print("Opening Website...")
    driver.get(TARGET_URL)
    time.sleep(3)

    # 3. Select Airline: Delta
    try:
        Select(driver.find_element(By.ID, "cboAirline")).select_by_value("DL")
        print("Selected Delta Air Lines.")
    
```

```

except Exception as e:
    print(f"Could not select Airline: {e}")
    driver.quit()
    exit()

time.sleep(2)

# --- LOOP YEARS ---
for year in range(START_YEAR, END_YEAR + 1):

    # --- LOOP MONTHS ---
    for month in range(1, 13):
        # Skip future dates (e.g. stop if current date is reached)
        if year == 2026 and month > 2:
            break

        # Calculate the last day (28, 30, or 31)
        last_day = calendar.monthrange(year, month)[1]
        month_name = calendar.month_name[month]

        print(f"Processing: {month_name} {year} (Days: {last_day})...")

        try:
            # --- STEP 1: SET YEARS FIRST ---
            # (Setting year first ensures leap years are handled correctly for Feb)
            Select(driver.find_element(By.ID, "stdateyear")).select_by_value(str(year))
            Select(driver.find_element(By.ID, "eddateyear")).select_by_value(str(year))

            # --- STEP 2: SET MONTHS ---
            Select(driver.find_element(By.ID, "stdatemon")).select_by_value(str(month))
            Select(driver.find_element(By.ID, "eddatemon")).select_by_value(str(month))

            # Give the site 1 second to populate the "Day" dropdown
            # (e.g., adding '31' if we switched from a 30-day month)
            time.sleep(1)

            # --- STEP 3: SET DAYS ---
            Select(driver.find_element(By.ID, "stdateday")).select_by_value("1")
            Select(driver.find_element(By.ID, "eddateday")).select_by_value(str(last_day))

            # --- STEP 4: SUBMIT ---
            driver.find_element(By.ID, "btnSubmit").click()

            # Wait for table to load
            time.sleep(2)

            # --- STEP 5: DOWNLOAD CSV ---
            # Check if the "Download" link exists (sometimes data is missing)
            try:
                driver.find_element(By.ID, "DL_CSV").click()

                # Wait for download
                time.sleep(2)

                # --- RENAME FILE ---
                files = [f for f in os.listdir(DOWNLOAD_DIR) if f.endswith(".csv")]
                if files:
                    full_paths = [os.path.join(DOWNLOAD_DIR, f) for f in files]
                    newest_file = max(full_paths, key=os.path.getctime)

                    # Target Name: Delta_2015_01.csv
                    new_name = os.path.join(DOWNLOAD_DIR, f"Delta_{year}_{month:02d}.csv")

                    if newest_file != new_name:

```

```

        if os.path.exists(new_name):
            os.remove(new_name)
        shutil.move(newest_file, new_name)
        print(f" -> Saved: Delta_{year}_{month:02d}.csv")

    except Exception as dl_error:
        print(f" -> No download link (Data likely missing for this month).")

    except Exception as e:
        print(f" -> Error on {month_name} {year}: {e}")
        continue

    # Pause before next loop
    time.sleep(1)

finally:
    driver.quit()
    print("Scraping Complete.")

#####
# Merging Statistics Data
#####

# --- CONFIGURATION ---
source_folder = "Airline_Monthly_Stat" # Folder with your downloaded files
output_file = "All_Airlines_Master_Data.csv"

# 1. Find all CSV files in the folder
files = glob.glob(os.path.join(source_folder, "*.csv"))

if not files:
    print(f"No files found in '{source_folder}'. Please check the folder name.")
else:
    print(f"Found {len(files)} files. Starting robust merge...")

    all_monthly_data = []

    for file in files:
        try:
            # --- ROBUST METADATA EXTRACTION ---
            with open(file, 'r', encoding='utf-8') as f:
                lines = [next(f) for _ in range(3)]

            airline_line = lines[1].strip() # Example: "Airline: American Airlines Inc. (AA)"
            time_line = lines[2].strip()   # Example: "Time Period: Thursday, January 1, 2015 to..."

            # 1. Extract BOTH Airline Name and Airline Code
            # This regex captures everything after "Airline: " up to the "(" as the Name,
            # and everything inside the "(" as the Code.
            match = re.search(r'Airline:\s*(.*?)\s*\((.*?)\)$', airline_line)
            if match:
                airline_name = match.group(1).strip() # "American Airlines Inc."
                airline_code = match.group(2).strip() # "AA"
            else:
                airline_name = "UNKNOWN"
                airline_code = "UNKNOWN"

            # 2. Extract Year and Month from the "Time Period" text
            date_part = time_line.split(" to ")[0].replace("Time Period: ", "").strip()
            parsed_date = datetime.strptime(date_part, "%A, %B %d, %Y")
            year = parsed_date.year
            month = parsed_date.month

            # --- EXTRACT TABLE 1: ALL FLIGHTS ---

```

```

df_all = pd.read_csv(file, skiprows=4, nrows=1)
df_all.columns = [f"All_{col.strip()}" for col in df_all.columns]

# --- EXTRACT TABLE 2: LATE FLIGHTS ---
df_late = pd.read_csv(file, skiprows=8, nrows=1)
df_late.columns = [f"Late_{col.strip()}" for col in df_late.columns]

# --- EXTRACT TABLE 3: CAUSE OF DELAY ---
df_cause = pd.read_csv(file, skiprows=12, nrows=1)
df_cause.columns = [f"Cause_{col.strip()}" for col in df_cause.columns]

# --- COMBINE TABLES ---
combined_row = pd.concat([
    df_all.reset_index(drop=True),
    df_late.reset_index(drop=True),
    df_cause.reset_index(drop=True)
], axis=1)

# --- ADD ACCURATE METADATA (WITH FULL NAME) ---
combined_row.insert(0, 'Airline_Name', airline_name)
combined_row.insert(1, 'Airline_Code', airline_code)
combined_row.insert(2, 'Year', year)
combined_row.insert(3, 'Month', month)

# --- CLEANUP ---
cols_to_drop = [c for c in combined_row.columns if 'Carrier' in c]
combined_row.drop(columns=cols_to_drop, inplace=True, errors='ignore')

all_monthly_data.append(combined_row)

except Exception as e:
    print(f"Error processing {os.path.basename(file)}: {e}")

# 2. CREATE MASTER DATAFRAME
if all_monthly_data:
    master_df = pd.concat(all_monthly_data, ignore_index=True)

# 3. SORT BY AIRLINE NAME, THEN DATE
master_df.sort_values(by=['Airline_Name', 'Year', 'Month'], inplace=True)

# 4. SAVE TO CSV
master_df.to_csv(output_file, index=False)
print(f"\nSuccess! Merged {len(master_df)} monthly records into '{output_file}'.")
print(master_df.head()) # Show first few rows
else:
    print("No valid data rows extracted.")

#####
# Merging Employee Data
#####

# --- CONFIGURATION ---
source_folder = "Employee" # Make sure this matches your folder name
output_file = "All_Airlines_Employee_Merged.csv" # Updated output name

# 1. Define Mappings

# Mapping dictionary for Entity codes
entity_map = {
    'A': 'Atlantic',
    'D': 'Domestic',
    'I': 'International',
    'L': 'Latin America',
    'P': 'Pacific',

```

```

'S': 'System'
}

# Mapping dictionary for Column Headers (Employee Data P-10)
column_header_map = {
    'YEAR': 'Year',
    'CARRIER_NAME': 'Carrier Name',
    'ENTITY': "Carrier's Operation Region",
    'GENERAL_MANAGE': 'General Managers',
    'PILOTS_COPILOTS': 'Pilots and Copilots',
    'OTHER_FLT_PERS': 'Other Flight Personnel',
    'PASS_GEN_SVC_ADMIN': 'Passenger/General Services & Administration',
    'MAINTENANCE': 'Maintenance Employees',
    'ARCFT_TRAF_HANDLING_GRP1': 'Aircraft Traffic Handling Group 1 Employees',
    'GEN_ARCFT_TRAF_HANDLING': 'General Aircraft Traffic Handling Employees',
    'AIRCRAFT_CONTROL': 'Aircraft Control Employees',
    'PASSENGER_HANDLING': 'Passenger Handling Employees',
    'CARGO_HANDLING': 'Cargo Handling Employees',
    'TRAINEES_INSTRUCTOR': 'Trainees and Instructor',
    'STATISTICAL': 'Statistical Employees',
    'TRAFFIC_SOLICITERS': 'Traffic Soliciters',
    'OTHER': 'Other Employees',
    'TRANSPORT_RELATED': 'Transport Related Employees',
    'TOTAL': 'Total Employees'
}

# 2. Find all CSV files in the folder
files = glob.glob(os.path.join(source_folder, "*.csv"))

if not files:
    print(f"No CSV files found in '{source_folder}'. Please check the folder path.")
else:
    print(f"Found {len(files)} files. Starting merge...")

    all_data = []

    for file in files:
        try:
            # Read the CSV file
            df = pd.read_csv(file)

            # Append all rows directly (No Delta filter)
            if not df.empty:
                all_data.append(df)
                print(f"-> Processed: {os.path.basename(file)} ({len(df)} rows)")

        except Exception as e:
            print(f"Error reading {file}: {e}")

    # 3. Concatenate all data
    if all_data:
        merged_df = pd.concat(all_data, ignore_index=True)

    # 4. Map the 'ENTITY' column to full descriptions
    if 'ENTITY' in merged_df.columns:
        # Replace the codes with descriptions using the map
        merged_df['ENTITY'] = merged_df['ENTITY'].map(entity_map).fillna(merged_df['ENTITY'])
        print("-> Entity column updated.")

    # 5. Apply Column Header Mapping (THIS WAS MISSING!)
    merged_df.rename(columns=column_header_map, inplace=True)
    print("-> Column headers renamed to match BTS descriptions.")

    # 6. Save the final merged file

```

```

merged_df.to_csv(output_file, index=False)
print(f"\nSuccess! Saved merged data to: {output_file}")
print(merged_df.head()) # Show first few rows

else:
    print("No data found in the files.")

#####
# Merging Financial Data
#####

# --- CONFIGURATION ---
source_folder = "Financial" # Folder containing your downloaded financial CSVs
output_file = "Airline_Financial_Data_Merged.csv"

# 1. Define Mappings

# Map for Region/Entity codes
# Based on BTS standard region codes
entity_value_map = {
    'A': 'Atlantic',
    'D': 'Domestic',
    'I': 'International',
    'L': 'Latin America',
    'P': 'Pacific',
    'S': 'System'
}

# Map for Column Headers (Variables -> Descriptions)
# Derived from BTS Schedule P-6 Descriptions
column_header_map = {
    'SALARIES_MGT': 'General Management Personnel',
    'SALARIES_FLIGHT': 'Flight Personnel',
    'SALARIES_MAINT': 'Maintenance Labor',
    'SALARIES_TRAFFIC': 'Aircraft and Traffic Handling Personnel',
    'SALARIES_OTHER': 'Other Personnel',
    'SALARIES': 'Total Salaries',
    'BENEFITS_PERSONNEL': 'Personnel Expense',
    'BENEFITS_PENSIONS': 'Employee Benefits and Pensions',
    'BENEFITS_PAYROLL': 'Payroll Taxes',
    'BENEFITS': 'Total Related Fringe Benefits',
    'SALARIES_BENEFITS': 'Total Salaries and Related Fringe Benefits',
    'AIRCRAFT_FUEL': 'Aircraft Fuel and Oil',
    'MAINT_MATERIAL': 'Maintenance Material',
    'FOOD': 'Passenger Food',
    'OTHER_MATERIALS': 'Other Materials',
    'MATERIALS_TOTAL': 'Total Materials',
    'ADVERTISING': 'Advertising and Other Promotions',
    'COMMUNICATION': 'Communication',
    'INSURANCE': 'Insurance',
    'OUTSIDE_EQUIP': 'Outside Flight Equipment Maintenance',
    'COMMISSIONS_PAX': 'Traffic Commissions - Passenger',
    'COMMISSIONS_CARGO': 'Traffic Commissions - Cargo',
    'OTHER_SERVICES': 'Other Services',
    'SERVICES_TOTAL': 'Total Services',
    'LANDING_FEES': 'Landing Fees',
    'RENTALS': 'Rentals',
    'DEPRECIATION': 'Depreciation',
    'AMORTIZATION': 'Amortization',
    'OTHER': 'Other Expenses',
    'TRANS_EXPENSE': 'Transport Related Expense',
    'OP_EXPENSE': 'Total Operating Expenses',
    'CARRIER_NAME': 'Carrier Name',

```

```

'REGION': "Carrier's Operation Region",
'YEAR': 'Year',
'QUARTER': 'Quarter'
}

# 2. Find all CSV files
files = glob.glob(os.path.join(source_folder, "*.csv"))

if not files:
    print(f"No CSV files found in '{source_folder}'. Please check the folder path.")
else:
    print(f"Found {len(files)} files. Starting merge...")

    all_data = []

# 3. Read and append all files (Added this loop back in)
for file in files:
    try:
        # Read the CSV file
        # Cast UNIQUE_CARRIER_ENTITY as string to fix domain format
        df = pd.read_csv(file, dtype={'UNIQUE_CARRIER_ENTITY': str})

        # Append all rows directly (No Delta filter)
        if not df.empty:
            all_data.append(df)
            print(f" -> Processed: {os.path.basename(file)} ({len(df)} rows)")

    except Exception as e:
        print(f"Error reading {file}: {e}")

# 4. Concatenate and Clean
if all_data:
    merged_df = pd.concat(all_data, ignore_index=True)

# 5. Filter to Delta relevant records prior to further cleaning
merged_df = merged_df[merged_df['CARRIER_NAME'] == 'Delta Air Lines Inc.']

# 6. Check for missing values post-filtering
missing = merged_df.isnull().sum()
missing = missing[missing > 0]
if not missing.empty:
    print("Missing values after Delta filter:")
    print(missing)

# 7. Apply Region Code Mapping (Changing the values inside the column)
# Note: In P-6 data, the column is often named 'REGION'
if 'REGION' in merged_df.columns:
    merged_df['REGION'] = merged_df['REGION'].map(entity_value_map).fillna(merged_df['REGION'])
    print(" -> Region codes updated to descriptions.")

# 8. Apply Column Header Mapping
merged_df.rename(columns=column_header_map, inplace=True)
print(" -> Column headers renamed to match BTS descriptions.")

# 9. Sort by Year and Quarter if available
sort_cols = [c for c in ['Year', 'Quarter'] if c in merged_df.columns]
if sort_cols:
    merged_df.sort_values(by=sort_cols, inplace=True)

# 10. Document Temporal coverage window
year_quarter = merged_df.groupby("Year")["Quarter"].nunique()
incomplete = year_quarter[year_quarter < 4]
if not incomplete.empty:
    print('Incomplete year(s) detected:')

```

```

        for yr, qcount in incomplete.items():
            print(f' {yr}: {qcount} of 4 quarters reported')
            print('Note: Exclude or document partial years',
                  'before subsequent analysis.')

# 11. Save to CSV
merged_df.to_csv(output_file, index=False)
print(f"\nSuccess! Saved merged data to: {output_file}")
print(merged_df.head())

else:
    print("No data found in the files.")

#####
# Data Dictionary (Type of Data)
#####

def create_comprehensive_dictionary():
    # Defining the full schema covering all columns across all merged datasets
    schema_data = [
        # =====
        # NOMINAL (Categorical, Labels, No real math order)
        # =====
        {'Dataset': 'Employee/Financial', 'Attribute': 'Carrier Name', 'Data_Type': 'String', 'Measurement_Scale': 'Nominal',
         'Description': 'Name of the airline operating the flights.'},
        {'Dataset': 'Employee/Financial', 'Attribute': 'Carrier's Operation Region', 'Data_Type': 'String',
         'Measurement_Scale': 'Nominal', 'Description': 'The geographic operation region (e.g., Domestic, Atlantic, Pacific).'},
        {'Dataset': 'Financial', 'Attribute': 'UNIQUE_CARRIER_ENTITY', 'Data_Type': 'String', 'Measurement_Scale':
         'Nominal', 'Description': 'Unique DOT identifier code for the airline/entity.'},
        {'Dataset': 'Financial', 'Attribute': 'CARRIER_GROUP', 'Data_Type': 'Integer', 'Measurement_Scale': 'Nominal',
         'Description': 'Legacy classification group code for the carrier.'},
        {'Dataset': 'Financial', 'Attribute': 'CARRIER_GROUP_NEW', 'Data_Type': 'Integer', 'Measurement_Scale':
         'Nominal', 'Description': 'Current classification group code for the carrier.'},

        # =====
        # ORDINAL (Categorical, Meaningful Order)
        # =====
        {'Dataset': 'Financial', 'Attribute': 'Quarter', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ordinal', 'Description':
         'Fiscal quarter of the year (1, 2, 3, or 4).'},
        {'Dataset': 'Flight Stats', 'Attribute': 'Month', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ordinal', 'Description':
         'Calendar month of the year (1 through 12).'},

        # =====
        # INTERVAL (Numeric, Exact Differences, No True Zero)
        # =====
        {'Dataset': 'All Datasets', 'Attribute': 'Year', 'Data_Type': 'Integer', 'Measurement_Scale': 'Interval', 'Description':
         'The calendar year of the report.'},

        # =====
        # RATIO (Numeric, True Zero meaning "None")
        # --- FLIGHT STATISTICS DATA ---
        # =====
        {'Dataset': 'Flight Stats', 'Attribute': 'All_Total Number', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
         'Description': 'Total number of scheduled flights.'},
        {'Dataset': 'Flight Stats', 'Attribute': 'All_Average Departure Delay (minutes)', 'Data_Type': 'Float',
         'Measurement_Scale': 'Ratio', 'Description': 'Average departure delay in minutes.'},
        {'Dataset': 'Flight Stats', 'Attribute': 'All_Average Taxi-Out Time (minutes)', 'Data_Type': 'Float',
         'Measurement_Scale': 'Ratio', 'Description': 'Average taxi-out time in minutes.'},
        {'Dataset': 'Flight Stats', 'Attribute': 'All_Average Scheduled Departure to Take-off (minutes)', 'Data_Type': 'Float',
         'Measurement_Scale': 'Ratio', 'Description': 'Average time from scheduled departure to actual take-off.'},
        {'Dataset': 'Flight Stats', 'Attribute': 'All_Average Arrival Delay (minutes)', 'Data_Type': 'Float',
         'Measurement_Scale': 'Ratio', 'Description': 'Average arrival delay in minutes.'},
        {'Dataset': 'Flight Stats', 'Attribute': 'All_Average Airborne Time (minutes)', 'Data_Type': 'Float',
         'Measurement_Scale': 'Ratio', 'Description': 'Average airborne flight time in minutes.'},
    ]

```

```

    {'Dataset': 'Flight Stats', 'Attribute': 'All_Average Taxi-In Time (minutes)', 'Data_Type': 'Float',
'Measurement_Scale': 'Ratio', 'Description': 'Average taxi-in time in minutes.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'All_Total Number Cancelled', 'Data_Type': 'Integer', 'Measurement_Scale':
'Ratio', 'Description': 'Total number of cancelled flights.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'All_Percent Flights Cancelled', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Percentage of total flights that were cancelled.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'All_Total Number Diverted', 'Data_Type': 'Integer', 'Measurement_Scale':
'Ratio', 'Description': 'Total number of diverted flights.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'All_Percent Flights Diverted', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Percentage of total flights that were diverted.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Late_Total Number', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Total number of late flights.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Late_Average Departure Delay (minutes)', 'Data_Type': 'Float',
'Measurement_Scale': 'Ratio', 'Description': 'Average departure delay for late flights only.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Late_Average Taxi-Out Time (minutes)', 'Data_Type': 'Float',
'Measurement_Scale': 'Ratio', 'Description': 'Average taxi-out time for late flights only.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Late_Average Scheduled Departure to Take-off (minutes)', 'Data_Type':
'Float', 'Measurement_Scale': 'Ratio', 'Description': 'Average time to take-off for late flights only.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Late_Average Arrival Delay (minutes)', 'Data_Type': 'Float',
'Measurement_Scale': 'Ratio', 'Description': 'Average arrival delay for late flights only.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Late_Average Airborne Time (minutes)', 'Data_Type': 'Float',
'Measurement_Scale': 'Ratio', 'Description': 'Average airborne time for late flights only.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Late_Average Taxi-In Time (minutes)', 'Data_Type': 'Float',
'Measurement_Scale': 'Ratio', 'Description': 'Average taxi-in time for late flights only.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Late_Percent Flights Late', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Percentage of flights classified as late.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Cause_Total Flights', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Total tracked flights for delay causes.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Cause_Diverted Flights Reaching Scheduled Destination', 'Data_Type':
'Integer', 'Measurement_Scale': 'Ratio', 'Description': 'Diverted flights that eventually reached their destination.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Cause_Diverted Flights', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Average delay caused by diversions.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Cause_Cancelled Flights', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Average ground time for cancelled flights.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Cause_Weather', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Average delay caused by weather.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Cause_National Aviation System', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Average delay caused by the National Aviation System.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Cause_Security', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Average delay caused by security issues.'},
    {'Dataset': 'Flight Stats', 'Attribute': 'Cause_Late Aircraft Arrival', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Average delay caused by previous late aircraft.'},

# =====
# RATIO (Numeric, True Zero meaning "None")
# --- EMPLOYEE COUNTS (P-10 Data) ---
# =====
    {'Dataset': 'Employee', 'Attribute': 'General Managers', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Headcount of General Managers.'},
    {'Dataset': 'Employee', 'Attribute': 'Pilots and Copilots', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Headcount of Pilots and Copilots.'},
    {'Dataset': 'Employee', 'Attribute': 'Other Flight Personnel', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Headcount of Other Flight Personnel.'},
    {'Dataset': 'Employee', 'Attribute': 'Passenger/General Services & Administration', 'Data_Type': 'Integer',
'Measurement_Scale': 'Ratio', 'Description': 'Headcount of G&A and Passenger Services staff.'},
    {'Dataset': 'Employee', 'Attribute': 'Maintenance Employees', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Headcount of Maintenance staff.'},
    {'Dataset': 'Employee', 'Attribute': 'Aircraft Traffic Handling Group 1 Employees', 'Data_Type': 'Integer',
'Measurement_Scale': 'Ratio', 'Description': 'Headcount for Traffic Handling Group 1.'},
    {'Dataset': 'Employee', 'Attribute': 'General Aircraft Traffic Handling Employees', 'Data_Type': 'Integer',
'Measurement_Scale': 'Ratio', 'Description': 'Headcount for General Aircraft Traffic Handling.'},
    {'Dataset': 'Employee', 'Attribute': 'Aircraft Control Employees', 'Data_Type': 'Integer', 'Measurement_Scale':
'Ratio', 'Description': 'Headcount of Aircraft Control Dispatchers.'},
    {'Dataset': 'Employee', 'Attribute': 'Passenger Handling Employees', 'Data_Type': 'Integer', 'Measurement_Scale':

```

```

'Ratio', 'Description': 'Headcount of Passenger Handling staff.'},
  {'Dataset': 'Employee', 'Attribute': 'Cargo Handling Employees', 'Data_Type': 'Integer', 'Measurement_Scale':
'Ratio', 'Description': 'Headcount of Cargo Handling staff.'},
  {'Dataset': 'Employee', 'Attribute': 'Trainees and Instructor', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Headcount of Trainees and Instructors.'},
  {'Dataset': 'Employee', 'Attribute': 'Statistical Employees', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Headcount of Statistical and recording staff.'},
  {'Dataset': 'Employee', 'Attribute': 'Traffic Soliciters', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Headcount of Traffic Soliciters / Sales.'},
  {'Dataset': 'Employee', 'Attribute': 'Other Employees', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Headcount of unclassified other employees.'},
  {'Dataset': 'Employee', 'Attribute': 'Transport Related Employees', 'Data_Type': 'Integer', 'Measurement_Scale':
'Ratio', 'Description': 'Headcount of Transport Related staff.'},
  {'Dataset': 'Employee', 'Attribute': 'Total Employees', 'Data_Type': 'Integer', 'Measurement_Scale': 'Ratio',
'Description': 'Sum of all employee categories.'},

# =====
# RATIO (Numeric, True Zero meaning "None")
# --- FINANCIAL METRICS (P-6 Data in $000s) ---
# =====
  {'Dataset': 'Financial', 'Attribute': 'General Management Personnel', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Cost of Management Salaries ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Flight Personnel', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Cost of Flight Personnel Salaries ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Maintenance Labor', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Cost of Maintenance Labor Salaries ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Aircraft and Traffic Handling Personnel', 'Data_Type': 'Float',
'Measurement_Scale': 'Ratio', 'Description': 'Cost of Aircraft Handling Salaries ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Other Personnel', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Cost of Other Salaries ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Total Salaries', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio', 'Description':
'Total Salary Cost ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Personnel Expense', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'General Personnel Expenses ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Employee Benefits and Pensions', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Cost of Benefits and Pensions ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Payroll Taxes', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio', 'Description':
'Payroll Taxes paid ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Total Related Fringe Benefits', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Total Benefits Cost ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Total Salaries and Related Fringe Benefits', 'Data_Type': 'Float',
'Measurement_Scale': 'Ratio', 'Description': 'Sum of Salaries and Benefits ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Aircraft Fuel and Oil', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Cost of Fuel and Oil ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Maintenance Material', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Cost of Maintenance Materials ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Passenger Food', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Cost of Food for Passengers ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Other Materials', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Cost of Other Materials ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Total Materials', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio', 'Description':
'Total Materials Cost ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Advertising and Other Promotions', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Cost of Advertising ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Communication', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Cost of Communications ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Insurance', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio', 'Description':
'Cost of Insurance ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Outside Flight Equipment Maintenance', 'Data_Type': 'Float',
'Measurement_Scale': 'Ratio', 'Description': 'Cost of Outside Maintenance ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Traffic Commissions - Passenger', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Cost of Passenger Commissions ($000s).'},
  {'Dataset': 'Financial', 'Attribute': 'Traffic Commissions - Cargo', 'Data_Type': 'Float', 'Measurement_Scale':
'Ratio', 'Description': 'Cost of Cargo Commissions ($000s).'},

```

```

        {'Dataset': 'Financial', 'Attribute': 'Other Services', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Cost of Other Services ($000s).'},
        {'Dataset': 'Financial', 'Attribute': 'Total Services', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio', 'Description':
'Total Services Cost ($000s).'},
        {'Dataset': 'Financial', 'Attribute': 'Landing Fees', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio', 'Description':
'Landing Fees paid ($000s).'},
        {'Dataset': 'Financial', 'Attribute': 'Rentals', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio', 'Description':
'Equipment/Space Rentals ($000s).'},
        {'Dataset': 'Financial', 'Attribute': 'Depreciation', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio', 'Description':
'Asset Depreciation ($000s).'},
        {'Dataset': 'Financial', 'Attribute': 'Amortization', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio', 'Description':
'Asset Amortization ($000s).'},
        {'Dataset': 'Financial', 'Attribute': 'Other Expenses', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Other Miscellaneous Expenses ($000s).'},
        {'Dataset': 'Financial', 'Attribute': 'Transport Related Expense', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Transport Related Expenses ($000s).'},
        {'Dataset': 'Financial', 'Attribute': 'Total Operating Expenses', 'Data_Type': 'Float', 'Measurement_Scale': 'Ratio',
'Description': 'Total Operating Expenses ($000s).'}
    ]

```

```

# Convert to DataFrame
df_dict = pd.DataFrame(schema_data)

```

```

# Save to CSV
output_filename = 'Master_Airline_Data_Dictionary.csv'
df_dict.to_csv(output_filename, index=False)

```

```

print(f"Data Dictionary successfully generated: {output_filename}")
print(f"Total attributes comprehensively mapped: {len(df_dict)}")

```

```

if __name__ == "__main__":
    create_comprehensive_dictionary()

```

```

def generate_uci_style_report(filepath, dataset_title, brief_description, output_filename):

```

```

    """Reads a CSV and generates a .names text report."""
    if not os.path.exists(filepath):
        print(f"File not found: {filepath}. Skipping...")
        return

```

```

    print(f"Processing {filepath}...")
    df = pd.read_csv(filepath)

```

```

    num_instances = df.shape[0]
    num_attributes = df.shape[1]

```

```

    report = []
    report.append(f"1. Title: {dataset_title}")
    report.append(f"2. Brief Description:\n {brief_description}")
    report.append(f"3. Number of Instances: {num_instances}")
    report.append(f"4. Number of Attributes: {num_attributes}")

```

```

    # --- MISSING VALUES ---

```

```

    report.append(f"5. Missing Attribute Values:")
    missing_total = df.isnull().sum().sum()
    if missing_total == 0:
        report.append(" None (Dataset is complete)")
    else:
        report.append(f" Total missing values across dataset: {missing_total}")
        for col in df.columns:
            missing = df[col].isnull().sum()
            if missing > 0:
                report.append(f" - {col}: {missing} missing ({missing/num_instances:.2%})")

```

```

# --- ATTRIBUTE INFORMATION (Type, Values, Outliers) ---
report.append(f"\n6. Attribute Information:")
for col in df.columns:
    dtype = str(df[col].dtype)

    if pd.api.types.is_numeric_dtype(df[col]):
        # Numeric analysis
        min_val = df[col].min()
        max_val = df[col].max()

        # Outlier calculation using IQR (Interquartile Range)
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers = ((df[col] < lower_bound) | (df[col] > upper_bound)).sum()

        attr_info = f" * {col}:"
        attr_info += f"\n    - Data Type: Continuous ({dtype})"
        attr_info += f"\n    - Values: Min: {min_val:.2f}, Max: {max_val:.2f}"
        attr_info += f"\n    - Outliers: {outliers} instances detected beyond 1.5*IQR bounds"
    else:
        # Categorical analysis
        unique_vals = df[col].dropna().unique()
        num_unique = len(unique_vals)

        if num_unique <= 10:
            vals_str = ", ".join(map(str, unique_vals))
        else:
            vals_str = f"{num_unique} unique categories (e.g., {' , '.join(map(str, unique_vals[:3]))})..."

        attr_info = f" * {col}:"
        attr_info += f"\n    - Data Type: Nominal/Categorical ({dtype})"
        attr_info += f"\n    - Values: [{vals_str}]"
        attr_info += f"\n    - Outliers: N/A for categorical attributes"

    report.append(attr_info)

# --- WRITE TO FILE ---
with open(output_filename, 'w') as f:
    f.write("\n".join(report))

print(f" -> Generated report: {output_filename}\n")

# =====
# EXECUTE THE GENERATOR FOR THE 3 DATASETS
# =====
if __name__ == "__main__":

    # 1. Flight Statistics
    generate_uci_style_report(
        filepath="All_Airlines_Master_Data.csv",
        dataset_title="USA Airlines Flight Performance Statistics (2015-2025)",
        brief_description="Contains monthly aggregated flight performance data including delays, cancellations, diversions, and causes of delays for USA Air Lines.",
        output_filename="Flight_Statistics.names"
    )

    # 2. Employee Data
    generate_uci_style_report(
        filepath="All_Airlines_Employee_Merged.csv",
        dataset_title="US Airlines Employee Demographics and Headcount (Form 41 Schedule P-10)",
        brief_description="Contains annual headcount data for US airlines, categorized by job function (e.g., Pilots,

```

```

Maintenance, Management) and operating region.",
    output_filename="Employee_Data.names"
)

# 3. Financial Data
generate_uci_style_report(
    filepath="Airline_Financial_Data_Merged.csv",
    dataset_title="US Airlines Financial and Operating Expenses (Form 41 Schedule P-6)",
    brief_description="Contains quarterly financial expense data for US airlines, including salaries, benefits, fuel,
maintenance, and total operating expenses.",
    output_filename="Financial_Data.names"
)

print("All .names files generated successfully!")

#####
# Outlier Detection on Delta-Filtered Data
#####

## Re-applies Tukey (1977) IQR-based outlier detection on Delta-only records to distinguish genuine
## operational anomalies from cross-carrier scale artifacts identified during the Investigating for Issues
## phase.

# CONFIGURATION
# Paths to the three merged CSV files
flight_file = 'All_Airlines_Master_Data.csv'
employee_file = 'All_Airlines_Employee_Merged.csv'
financial_file = 'Airline_Financial_Data_Merged.csv'

# Delta filter values (as they appear in each dataset)
delta_filters = {
    'Flight Statistics': {
        'file': flight_file,
        'filter_col': 'Airline_Name',
        'filter_val': 'Delta Air Lines Inc.'
    },
    'Employee Data': {
        'file': employee_file,
        'filter_col': 'Carrier Name',
        'filter_val': 'Delta Air Lines Inc.'
    },
    'Financial Data': {
        'file': financial_file,
        'filter_col': 'Carrier Name',
        'filter_val': 'Delta Air Lines Inc.'
    }
}

def detect_outliers_iqr(df, dataset_name):
    """
    Re-applies Tukey (1977) IQR-based outlier detection
    on Delta-filtered records. Flags and documents outliers
    rather than removing them.
    """
    numeric_cols = df.select_dtypes(include='number').columns
    results = []

    for col in numeric_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR

```

```

upper_bound = Q3 + 1.5 * IQR

outlier_mask = (df[col] < lower_bound) | (df[col] > upper_bound)
n_outliers = outlier_mask.sum()

if n_outliers > 0:
    results.append({
        'Dataset': dataset_name,
        'Feature': col,
        'N_Outliers': n_outliers,
        'Lower_Bound': round(lower_bound, 2),
        'Upper_Bound': round(upper_bound, 2),
        'Min_Value': round(df[col].min(), 2),
        'Max_Value': round(df[col].max(), 2)
    })

return results

# RUN OUTLIER DETECTION ACROSS ALL THREE DATASETS
all_results = []

for name, config in delta_filters.items():
    try:
        df = pd.read_csv(config['file'])
        df_delta = df[df[config['filter_col']] == config['filter_val']]
        print(f'{name}: {len(df_delta)} Delta records (filtered from {len(df)} total)')

        results = detect_outliers_iqr(df_delta, name)
        all_results.extend(results)

    except FileNotFoundError:
        print(f'{name}: File not found ({config["file"]}). Skipping.')
    except Exception as e:
        print(f'{name}: Error - {e}')

# SUMMARY
if all_results:
    summary_df = pd.DataFrame(all_results)
    print(f'\n--- Outlier Summary (Delta-Filtered, IQR Method) ---')
    print(f'Total features with outliers: {len(summary_df)}')
    print()
    for dataset in summary_df['Dataset'].unique():
        subset = summary_df[summary_df['Dataset'] == dataset]
        print(f'{dataset}: {len(subset)} feature(s) with outliers')
        for _, row in subset.iterrows():
            print(f' {row["Feature"]}: {row["N_Outliers"]} outlier(s) '
                  f' [bounds: {row["Lower_Bound"]} to {row["Upper_Bound"]}], '
                  f'actual range: {row["Min_Value"]} to {row["Max_Value"]}')
        print()

    # Save annotated summary for documentation
    summary_df.to_csv('Delta_Outlier_Summary.csv', index=False)
    print('Saved: Delta_Outlier_Summary.csv')
else:
    print('\nNo outliers detected in Delta-filtered records.')

print('\nNote: Flagged records are preserved and annotated rather than removed,'
      ' consistent with the project objective of detecting peak-season disruptions.')

```

APPENDIX B

Delta-Filtered Outlier Summary (IQR Method)

Dataset	Feature	# Outliers
Flight Statistics	All_Total Number	11
Flight Statistics	All_Average Departure Delay (minutes)	1
Flight Statistics	All_Average Taxi-Out Time (minutes)	7
Flight Statistics	All_Average Scheduled Departure to Takeoff (minutes)	1
Flight Statistics	All_Average Arrival Delay (minutes)	1
Flight Statistics	All_Average Taxi-In Time (minutes)	2
Flight Statistics	All_Total Number Cancelled	11
Flight Statistics	All_Percent Flights Cancelled	10
Flight Statistics	All_Total Number Diverted	6
Flight Statistics	All_Percent Flights Diverted	2
Flight Statistics	Late_Total Number	3
Flight Statistics	Late_Average Departure Delay (minutes)	3
Flight Statistics	Late_Average Taxi-Out Time (minutes)	1
Flight Statistics	Late_Average Scheduled Departure to Takeoff (minutes)	1
Flight Statistics	Late_Average Arrival Delay (minutes)	3
Flight Statistics	Late_Average Airborne Time (minutes)	1
Flight Statistics	Late_Average Taxi-In Time (minutes)	2
Flight Statistics	Late_Percent Flights Late	2
Flight Statistics	Cause_Total Flights	11
Flight Statistics	Cause_Diverted Flights Reaching Scheduled Destination	6
Flight Statistics	Cause_Diverted Flights	4
Flight Statistics	Cause_Cancelled Flights	4
Flight Statistics	Cause_Weather	9
Flight Statistics	Cause_Security	12
Flight Statistics	Cause_Late Aircraft Arrival	4
Employee Data	General Managers	8
Employee Data	Pilots and Copilots	10
Employee Data	Passenger/General Services & Administration	10
Employee Data	Maintenance Employees	11
Employee Data	General Aircraft Traffic Handling Employees	7
Employee Data	Aircraft Control Employees	9
Employee Data	Passenger Handling Employees	11
Employee Data	Cargo Handling Employees	10

Employee Data	Trainees and Instructor	3
Employee Data	Statistical Employees	10
Employee Data	Traffic Soliciters	9
Employee Data	Other Employees	8
Employee Data	Total Employees	10
Financial Data	General Management Personnel	23
Financial Data	Flight Personnel	11
Financial Data	Maintenance Labor	17
Financial Data	Aircraft and Traffic Handling Personnel	22
Financial Data	Other Personnel	15
Financial Data	Total Salaries	10
Financial Data	Personnel Expense	12
Financial Data	Employee Benefits and Pensions	12
Financial Data	Payroll Taxes	33
Financial Data	Total Related Fringe Benefits	12
Financial Data	Total Salaries and Related Fringe Benefits	10
Financial Data	Aircraft Fuel and Oil	10
Financial Data	Maintenance Material	34
Financial Data	Passenger Food	15
Financial Data	Other Materials	30
Financial Data	Total Materials	15
Financial Data	Advertising and Other Promotions	20
Financial Data	Communication	27
Financial Data	Insurance	24
Financial Data	Outside Flight Equipment Maintenance	42
Financial Data	Traffic Commissions - Passenger	39
Financial Data	Traffic Commissions - Cargo	41
Financial Data	Other Services	14
Financial Data	Total Services	21
Financial Data	Landing Fees	11
Financial Data	Rentals	8
Financial Data	Depreciation	37
Financial Data	Amortization	29
Financial Data	Other Expenses	5

Financial Data	Transport Related Expense	47
Financial Data	Total Operating Expenses	22